

QUIZARD FEATURES STORYBOOK

Master Architecture, Gameplay Mechanics, and
Tournament Systems

A Comprehensive Technical Specification and Narrative Guide for Decision
Makers, Product Managers, and Backend Software Engineers

Core System Pillars Covered

- **Authentication & Identity Lifecycle:** MSISDN regex normalization, carrier billing routing, and token invalidation
- **Dynamic Rules Governance:** Server-driven timers (180s), 60-question sets, and 2-attempt daily limits
- **High-Speed Question Engine:** Sub-50ms ID sampling, difficulty weighting, and anti-memorization exclusions
- **Military-Grade Cryptography:** AES-256-CBC payload encryption and anti-cheat velocity verification
- **Live Leaderboards & Tie-Breaking:** Real-time SQL window ranking and strict time-based tie resolution
- **Score-Driven Performance Bands:** 6 score-driven tiers (0 to 60) with asymmetric delta adjustments
- **XP Ledger & Daily Streaks:** Daily streak consistency (+20 XP/day) with giant +500 XP Day 15 surge
- **User Personalization & Velocity:** Data-backed retention nudges ($3 \times / 5.6 \times$ gap) and $\uparrow / \downarrow$ rank arrows
- **Multi-Modal Quiz Formats:** Image, audio snippet, video moments, and Bengali fill-in-the-blank
- **Adaptive Match Governance:** Dual-track inclusive standard vs opt-in Grandmaster Blitz mode
- **Real-Time WebSockets Architecture:** Sub-15ms SignalR hubs with Redis Pub/Sub for live head-to-head duels
- **AI Voice Q&A & RAG Engine:** NCTB/BCS vector retrieval, Small Local Model (7B) vs Cloud LLM evaluation
- **High-Velocity Telemetry Gateway:** Dedicated FastAPI microservice (cms.quizard.live) on MongoDB & Redis
- **CMS Operations & Telco Billing:** Question approval lifecycles, Wordly, Speed Math, DOB, and bKash passes
- **Financial Reporting & BI Engine:** Multi-tier winner audits, automated bKash B2C payouts, and revenue analytics
- **Clean Architecture Migration:** Transitioning from Django REST to high-throughput .NET 10 on PostgreSQL

Author: Engineering Architecture Team

Architecture Evolution: Quizard-Backend (Django MySQL $\rightarrow$.NET 10 PostgreSQL) $\longleftrightarrow$ Next.js 16 Frontend $\longleftrightarrow$ Quizard-API (FastAPI / MongoDB)

Database Infrastructure: PostgreSQL Enterprise Cluster + Telemetry MongoDB Cluster (Migrated from MySQL)

Date: August 2026

Contents

Executive Overview: The Quizard Platform	5
1 Authentication, Identity Lifecycle, Multi-Tenant Sessions, and Logout	6
1.1 MSISDN Normalization and Identity Provisioning	6
1.1.1 Just-in-Time (JIT) Tenant Provisioning	7
1.2 Multi-Tenant Subscription Entitlements and Direct Billing Routing	7
1.2.1 Multi-Service Entitlement Flags	8
1.3 Two-Factor SMS OTP Verification Workflow	8
1.4 Graceful Logout and Security Token Invalidation	9
1.4.1 1. Server-Side Cryptographic Token Destruction	9
1.4.2 2. Client-Side State and Storage Sanitization	9
1.5 Summary of Authentication Endpoints	10
2 The Rules Engine: Game Governance, Timers, and Attempt Limits	11
2.1 Why Dynamic Rules Matter	11
2.2 The Player Journey: Rahim Checks Today's Tournament Rules	12
2.2.1 Morning Setup in Dhaka	12
2.2.2 Enforcing the Daily Attempt Cap	12
3 The Question Engine: Selection, Difficulty Weighting, and Real-Time Delivery	14
3.1 The Architecture of the Question Bank	14
3.2 The Question Selection Algorithm	14
3.2.1 Why Fast ID Sampling Beats Full Database Sorting	15
3.3 The Player Journey: Karim Plays a 60-Question Rapid Match	15
3.3.1 Session Launch in Chittagong	15
3.3.2 Racing Against the 180-Second Clock	16
4 The Shield of Integrity: Cryptography, Anti-Cheat, and Score Persistence	17
4.1 The End-to-End Cryptographic Handshake	17
4.2 The Multi-Layer Anti-Cheat Verification Suite	18
4.3 Two Divergent Journeys: Salma vs. Tanvir	18
4.3.1 Salma's Legitimate Victory (59/60 in 150 Seconds)	18
4.3.2 Tanvir's Automated Exploit Attempt (Flagged & Blocked)	19
5 The Hall of Champions: Live Leaderboards and Tournament Consolidation	20
5.1 The Real-Time Daily Leaderboard Engine	20
5.1.1 Strict Tie-Breaking Hierarchy	21
5.2 The Overnight Daily Verification Pipeline (04:30 AM Cron)	21
5.3 The Weekly Tournament Championship (Thursday 05:30 AM)	21
5.3.1 The 7-Day Rolling Tournament Cycle	21
5.3.2 Stored Procedure Architecture	22

5.4	The Player Journey: Ayesha Wins the Weekly Championship	22
6	The Performance Rating Architecture: 6 Score Bands, Dynamic Deltas, and Progression Modeling	24
6.1	The 6 Integer Performance Bands (0 to 60 Scale)	24
6.2	The Integer Dynamic Rating Update Formula	24
6.2.1	Core Principles of the Integer Engine	25
6.3	Step-by-Step Scenarios: The Player at Rating 34	25
6.3.1	Detailed Analysis of Each Case	25
6.4	Multi-Round Lifecycle Simulation (Pure Integers 0 to 60)	26
6.5	Database Schema and API Architecture	26
6.5.1	1. PostgreSQL Historical Rating Table	26
6.5.2	2. REST Endpoints Specification	27
7	The XP Ledger, Daily Streaks, and Milestone Achievement System	28
7.1	Core Experience Point (XP) Earning Mechanics	28
7.2	Daily Streaks with the Giant 15th-Day Surge	28
7.2.1	Daily Streak Rule	28
7.2.2	The 15-Day Milestone Progression Matrix	30
7.3	Milestone Achievement Badges Shelf	30
7.4	Database Schema and API Architecture	30
7.4.1	1. PostgreSQL Immutable XP Ledger Table	30
7.4.2	2. REST Endpoints Specification	30
7.5	Real-World Scenario: Sabina's 15-Day Streak Journey	31
8	User Personalization Engine, Rank Velocity, and Retention Nudges	32
8.1	The Retention Crisis and Empirical Evidence	32
8.2	The Multi-Tier Profile Personalization Engine	33
8.2.1	1. Missing Rewards Nudge (Unsubscribed Players)	33
8.2.2	2. Cross-Game Ecosystem Discovery Nudge (Single-Game Players)	33
8.2.3	3. Inactivity Win-Back Nudge (Lapsed 24h+ Players)	33
8.3	Real-Time Leaderboard Rank Velocity Indicators	33
8.3.1	Velocity and Points-to-Top-10 Mathematics	34
8.4	PostgreSQL Schema API Implementation	34
9	Multi-Modal Quiz Formats: Visual, Audio, Video, and Fill-in-the-Blank	35
9.1	The Format Fatigue Diagnosis (Query H)	35
9.2	The 4 Core Multi-Modal Question Archetypes	35
9.2.1	1. Image-Based Identification (<code>image_mcq</code>)	36
9.2.2	2. Audio Snippet Challenges (<code>audio_clip</code>)	36
9.2.3	3. Short Video Moments (<code>video_clip</code>)	36
9.2.4	4. Fill-in-the-Blank with Virtual Tile Keyboard (<code>fill_blank</code>)	36
9.3	Sub-50ms Mobile Asset Preloading Architecture	36
9.4	Database Schema and JSON Question Payloads	36
10	Adaptive Difficulty and High-Performer Retention Dynamics	38
10.1	The High-Performer Dilemma (Queries B & D)	38
10.2	The Dual-Track Difficulty Architecture	39
10.3	Dynamic In-Round Question Adaptation (Streak Escalation)	39
10.3.1	Graceful Difficulty De-Escalation	39
10.4	Database Configuration Implementation	39

11 Real-Time WebSockets Architecture and Live Head-to-Head Duels	41
11.1 HTTP Polling vs WebSockets Architectural Evaluation	41
11.2 Distributed WebSocket Hub with Redis Pub/Sub Backplane	42
11.3 .NET 10 SignalR Hub Implementation	42
12 AI Voice QA, Bengali NLP, and Retrieval-Augmented Generation (RAG)	44
12.1 The Retrieval-Augmented Generation (RAG) Architecture	44
12.2 Model Evaluation: Small Local Models vs Large Cloud Models	45
12.2.1 Architectural Recommendation: The Hybrid AI Strategy	45
12.3 The Multi-Modal Voice Processing Pipeline	45
12.4 PostgreSQL Vector Schema Implementation	45
13 The Gateway of Speed: The FastAPI Microservice and CMS Engine	47
13.1 The Need for a Dedicated High-Velocity Microservice	47
13.2 Core Responsibilities of Quizard-API	48
13.3 Real-World Stories and Concrete Examples	48
13.3.1 Story 1: Rahim's Campaign Click and Asynchronous Telemetry	48
13.3.2 Story 2: Marketing Executives Analyze Hourly Campaign ROI	49
13.3.3 Story 3: Karim Fetches Instant Cached Rules	49
14 The Master Ecosystem: CMS Operations, Multi-Game Engines, and Telco Billing	51
14.1 The CMS Question Authoring and Approval Pipeline	51
14.1.1 Data Model of a CMS Question Entity	51
14.2 Admin Governance and Role-Based Access Control (RBAC)	52
14.3 The Multi-Game Suite: Beyond Rapid Trivia	52
14.3.1 1. Wordly: The Daily Bengali & English Word Puzzle	52
14.3.2 2. Speed Math and Addition Sprint	52
14.3.3 3. Memory Match and Sudoku Challenges	52
14.4 Direct Operator Billing (DOB) and Telco Subscriptions	53
14.5 Financial Operations, Referrals, and Prize Disbursements	53
14.6 Ad Network Attribution and Campaign Tracking	53
14.7 Real-World Stories: Behind the Scenes	54
14.7.1 Story 1: Farhana Prepares the National BCS Tournament Questions	54
14.7.2 Story 2: Rupa Wins the Daily Wordly Challenge in Sylhet	54
14.7.3 Story 3: Monir Audits and Disburses Weekly Prizes	54
15 The Financial and Analytics Hub: Reporting Engine, Payouts, and Business Intelligence	55
15.1 The Architecture of the Reporting Engine	55
15.2 Core Modules of the Reporting Suite	55
15.2.1 1. Daily and Weekly Tournament Winner Reports	55
15.2.2 2. Micro-Prize Tier Reporting (5 TK and 10 TK Bounties)	56
15.2.3 3. Wordly Word Puzzle Winner Analytics	56
15.2.4 4. Referral Tracking and Bounties (PrizeOnReferral)	56
15.2.5 5. Churn-Back and Win-Back Analytics (ChurnBackReportViewSet)	56
15.2.6 6. Real-Time Revenue and Charging Graphs (apps.allReportGraph)	56
15.3 Real-World Scenario: Tanima Audits and Executes Weekly Payouts	56
16 Architecture Blueprint: Clean Architecture and Stored Procedure Decoupling	58
16.1 Why Stored Procedures Are an Anti-Pattern for Live Backends	58
16.2 The Architectural Separation Rule: What to Move vs What Stays	59
16.2.1 1. Procedures That MUST Be Migrated to .NET 10 Clean Architecture	59
16.2.2 2. Operations That Appropriately Remain in PostgreSQL (Batch Crons)	59

16.3 The .NET 10 Clean Architecture Structure	59
16.4 Comprehensive Directory of Stored Procedures to Remove	60
16.5 Clean Architecture Implementation Example: Profile Summary	62
17 Technical Glossary and Authoritative Reference Directories	64
17.1 Authoritative Event and Service ID Directory	64
17.2 Authoritative Multi-Tenant Portal Directory (PortalType)	64
17.3 Glossary of Core Domain Entities	64

Executive Overview: The Quizard Platform

Platform Mission

Quizard is Bangladesh's premier real-time interactive quiz and tournament platform, engaging hundreds of thousands of concurrent players across web and telecommunication partner channels (bKash, Banglalink, Grameenphone, Robi).

To ensure absolute competitive integrity, fair prize distribution, and instantaneous responsiveness, Quizard combines server-authoritative game governance, sub-50ms dynamic question generation, end-to-end payload cryptography, and automated tournament consolidation.

The End-to-End System Architecture

The diagram below illustrates the comprehensive lifecycle of a player's journey through Quizard:

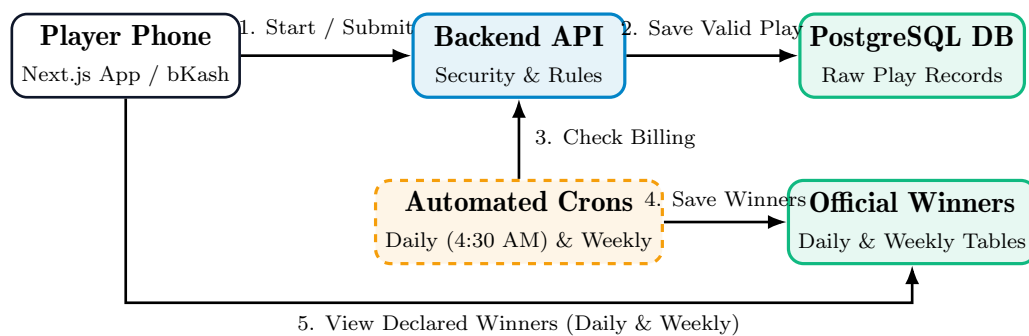


Figure 1: Quizard End-to-End Competition and Winner Lifecycle

Chapter 1

Authentication, Identity Lifecycle, Multi-Tenant Sessions, and Logout

The Story of Nadia's Seamless Login and Secure Sign-Out

It is 08:30 PM on a Friday evening in Chittagong. Nadia, an aspiring civil service candidate and regular tournament player, opens the Quizard application on her Android smartphone. She enters her mobile number to compete in the daily tournament. Behind the glass screen, a high-speed identity orchestration takes place in less than **35 milliseconds**:

1. Her phone number is normalized to a canonical format regardless of how she typed it (01711..., +8801711..., or 8801711...).
2. The backend detects that Nadia is a registered player on ****Portal 15 (bKash/Quizard Core)****.
3. Her carrier subscription status across all game categories (Daily, Tournament, BCS Mind Masters, Sports) is evaluated in parallel.
4. A cryptographic authentication token is issued, granting her immediate access to her profile, rank velocity, and tournament lobbies.
5. When Nadia finishes her tournament rounds and chooses to sign out before handing her phone to her sibling, a single tap triggers a secure token invalidation that guarantees no subsequent unauthorized API requests can be made from cached credentials.

1.1 MSISDN Normalization and Identity Provisioning

In Bangladesh, mobile phone numbers (MSISDNs) are entered in various formats across mobile browsers, SMS links, and native webviews:

- Full International Format: +8801711000001
- Country Code without Prefix: 8801711000001
- Standard National Format: 01711000001
- Local 10-digit Shorthand: 1711000001

To prevent duplicate accounts and ensure deterministic user lookups, the authentication engine executes strict MSISDN sanitization before touching the database:

Listing 1.1: MSISDN Normalization Logic in Django / .NET 10

```

1 def normalize_msisdn(msisdn: str) -> str:
2     # 1. Strip all non-digit characters (spaces, dashes, plus signs)
3     cleaned = re.sub(r"\D", "", msisdn)
4
5     # 2. Strip leading international country code (880)
6     if cleaned.startswith("880"):
7         cleaned = cleaned[3:]
8
9     # 3. Strip leading national zero (0)
10    if cleaned.startswith("0"):
11        cleaned = cleaned[1:]
12
13    return cleaned # Returns canonical 10-digit string: e.g.,
                    '1711000001'

```

1.1.1 Just-in-Time (JIT) Tenant Provisioning

When a player logs in via POST /api/login/:

1. **Global User Check (auth_user):** The engine checks if the canonical username exists. If not, a new user entity is provisioned automatically with a secure default state.
2. **Tenant Profile Check (ParticipantDetail):** Quizard checks for a tenant profile matching the player's `user_id` and the incoming `portal_id` (e.g., Portal 15 for bKash Web, Portal 16 for Banglalink, Portal 17 for Grameenphone, Portal 21 for Robi).
3. **Referral & Promo Attribution:** If a promotional campaign code (`promo=XYZ`) or referral token is present in the query parameters, it is linked to the participant's record to unlock promotional passes.

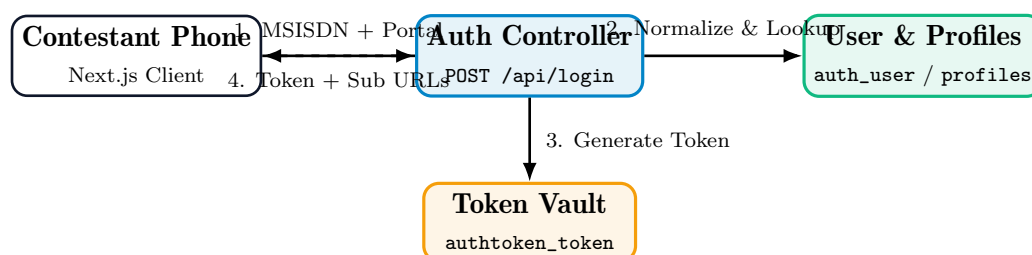


Figure 1.1: End-to-End Authentication and Multi-Tenant Provisioning Workflow

1.2 Multi-Tenant Subscription Entitlements and Direct Billing Routing

Upon successful identity lookup, the login response dynamically constructs the player's access rights and operator billing redirection paths based on their active carrier tenant:

Portal ID	Operator Channel	Billing Gateway URL (subURL)	Entitlement Resolution
15	bKash / Quizard	https://cms.quizard.live/subscription/create/	Validates daily bKash tokenized MFS subscriptions.
16	Banglalink (BL)	https://cms.quizard.live/bl/user_subscription	Checks Banglalink Applink direct carrier billing status.
17	Grameenphone (GP)	https://cms.quizard.live/gp/gp_user_subscription/	Verifies GP Digimart DOB active registration status.
18	Wordly Standalone	https://cms.quizard.live/wordly/subscription/create/	Checks dedicated Wordly puzzle subscription status.
19	EngageX Promo	<i>(Empty - Always Subscribed)</i>	Automatically grants active access across all categories.
20	bKash AppInApp	https://cms.quizard.live/appinapp/subscription/create/	Validates embedded webview session in bKash app.
21	Robi ShadhinMusic	<i>(Partner Telco API)</i>	Direct carrier postback validation with Robi engine.

Table 1.1: Dynamic Carrier Billing and Subscription Resolution Matrix

1.2.1 Multi-Service Entitlement Flags

Rather than a single binary flag, the login response provides fine-grained authorization across Quizard's game portfolio:

Listing 1.2: Comprehensive Login Response Payload

```

1 {
2   "message": "Login successful",
3   "token": "4f8a9b2c1d3e5f7a8b9c0d1e2f3a4b5c6d7e8f9a",
4   "is_verified": true,
5   "subURL": "https://cms.quizard.live/subscription/create/",
6   "isSubscribe": true,
7   "isSubscribeTournament": true,
8   "isSubscribeBcs": false,
9   "isSubscribeSports": true,
10  "isSubscribeWordly": false,
11  "isSubscribeSpellingBee": false,
12  "isSubscribeMemoryMatch": false,
13  "isSubscribeSudokuEasy": false
14 }
```

1.3 Two-Factor SMS OTP Verification Workflow

For high-security portals or new mobile registrations requiring explicit carrier verification, Quizard integrates an automated SMS OTP gateway:

1. **OTP Dispatch:** A cryptographically secure 6-digit random code is generated, stored in `ParticipantDetail.otp`, and dispatched via the Telecom SMS Gateway:

$$\text{OTP} \in [100000, 999999]$$

2. **Verification Request** (POST `/api/otpverify_api/`): The client submits the MSISDN, OTP, and Company ID:

Listing 1.3: OTP Verification Request

```

1 {
2   "msisdn": "01711000001",
3   "otp": "482910",
4   "company_id": 15
5 }
```

3. **Validation and Account Activation:** The server compares the submitted OTP against the database record. Upon a match, `ParticipantDetail.is_verified` is set to `true`, the temporary OTP is purged, and the authenticated session token is released.

1.4 Graceful Logout and Security Token Invalidation

In multi-user environments (such as family members sharing a tablet or smartphone), incomplete logouts represent a critical security vulnerability where cached browser tokens could allow unauthorized tournament attempts or prize theft.

Quizard implements **Dual-Boundary Token Invalidation**:

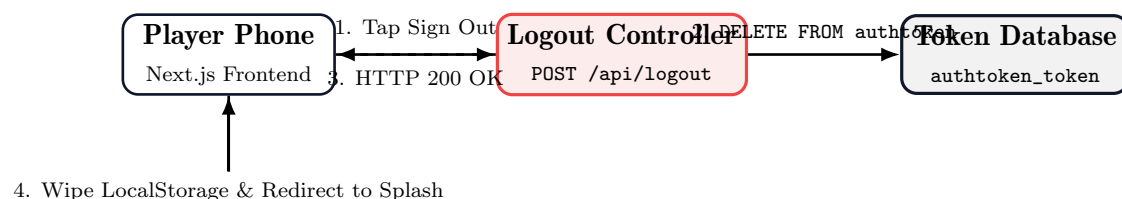


Figure 1.2: Dual-Boundary Logout and Cryptographic Token Destruction

1.4.1 1. Server-Side Cryptographic Token Destruction

When the player triggers sign-out, the client sends an authenticated request to `POST /api/logout/` carrying the active header:

Authorization: Token 4f8a9b2c1d3e5f7a8b9c0d1e2f3a4b5c6d7e8f9a

The server instantly executes:

Listing 1.4: Server-Side Token Deletion in .NET 10 AuthService

```

1 public async Task LogoutAsync(string tokenKey, CancellationToken ct =
2   default)
3 {
4     // Instantly remove token from the database
5     await _dbContext.AuthTokens
6       .Where(t => t.Key == tokenKey)
7       .ExecuteDeleteAsync(ct);
8
9     _logger.LogInformation("Security token {TokenPrefix}...
10    successfully invalidated.",
11      tokenKey[..6]);
12 }
  
```

Any subsequent attempt to reuse the old token immediately fails with an **HTTP 401 Unauthorized** error.

1.4.2 2. Client-Side State and Storage Sanitization

Upon receiving the HTTP 200 response from the logout endpoint, the Next.js frontend clears all client-side authentication artifacts:

- Clears `localStorage.removeItem('token')` and `localStorage.removeItem('playerData')`.
- Resets the global React authentication state and Zustand stores to default unauthenticated values.

- Redirects the browser to the public splash/landing view, preventing backward navigation via browser cache.

1.5 Summary of Authentication Endpoints

The table below summarizes the core identity endpoints available in the Quizard API suite:

HTTP Method	Endpoint Route	Auth Level	Functional Purpose
POST	/api/login/	Anonymous	MSISDN authentication, JIT provisioning, and entitlement resolution.
POST	/api/login_platform/	Anonymous	Platform-specific partner login with tenant-scoped URLs.
POST	/api/otpverify_api/	Anonymous	Validates 6-digit SMS OTP code for unverified subscribers.
POST	/api/customlogin/	Anonymous	Username/password administrative login for CMS analytics dashboard.
POST	/api/logout/	Authenticated	Destroys active token in database and terminates user session.

Table 1.2: Comprehensive Identity and Authentication Endpoint Catalog

Chapter Key Takeaways

- **Deterministic Identity:** MSISDN normalization eliminates duplicate phone representations and guarantees consistent account mapping.
- **Multi-Tenant Redirection:** Dynamic subURL resolution routes contestants seamlessly to their respective telecom carrier billing gateways.
- **Dual-Boundary Logout:** Simultaneous server-side token destruction and client-side storage wiping ensure complete protection against unauthorized session hijacking.

Chapter 2

The Rules Engine: Game Governance, Timers, and Attempt Limits

The Rules Engine in Brief

Before a single question appears on a player's phone screen, the Quizard backend consults its **Rules Engine**. The Rules Engine acts as the tournament constitution: it dynamically sets how many questions each player receives (e.g., 60 questions), how many seconds they have to answer them (e.g., 180 seconds / 3 minutes), how many attempts they are allowed per day (e.g., exactly 2 rounds per day), and what reward prizes are at stake.

2.1 Why Dynamic Rules Matter

In traditional mobile games, quiz rules and timers are often hardcoded directly into the phone app. If tournament organizers want to change a quiz duration from 3 minutes to 2 minutes for a special holiday tournament, they would normally have to build and publish a brand new app update to the Google Play Store and Apple App Store.

Quizard solves this through its **Backend-Driven Rules Engine** (mapped in Django to `apps.cms.models.Rules` and in .NET to the `Rule` domain entity). Every aspect of gameplay is determined on the server in real time:

- **Allocated Time** (`allocated_time`): Total gameplay window in seconds (e.g., 180 seconds = 3 minutes).
- **Question Set Count** (`question_set`): Total questions served per quiz round (e.g., 60 questions).
- **Daily Round Limit** (`round`): Maximum allowed plays per calendar day (e.g., 2 attempts per day).
- **Passing Score** (`passing_score`): Minimum correct answers required to qualify for daily rewards (e.g., 20 correct answers).
- **Category Distribution** (`category`): Array of approved topic IDs (General Knowledge, BCS Preparation, Sports, Science, Islamic History).
- **Reward Prize Pool** (`winningPrize`): Daily, weekly, or special event cash prizes (e.g., 5,000 BDT to 25,000 BDT).

2.2 The Player Journey: Rahim Checks Today's Tournament Rules

2.2.1 Morning Setup in Dhaka

It is 08:30 AM in Dhaka. Rahim, an avid university student and competitive quiz enthusiast, opens the Quizard mobile web application on his smartphone.

Before Rahim sees the "Start Quiz" button, his phone sends a background request to the backend:

GET /api/rules/?portal=15&events=34



Figure 2.1: The Dynamic Rules Retrieval Lifecycle

The backend retrieves the active configuration for Event 34 on Portal 15 (bKash/Quizard Live) and delivers the JSON configuration:

Listing 2.1: Rules Engine JSON Payload for 60-Question Tournament

```
1 {
2   "id": 34,
3   "events": "Islamic Daily Quiz Challenge",
4   "category": [1, 2, 3, 4, 5],
5   "question_set": 60,
6   "allocated_time": 180,
7   "round": 2,
8   "passing_score": 20,
9   "winningPrize": "10,000 BDT",
10  "status": "active",
11  "company": 15
12 }
```

2.2.2 Enforcing the Daily Attempt Cap

Rahim knows he has ****2 attempts per day**** for this tournament.

1. **First Attempt at 09:00 AM:** Rahim plays his first round. He answers 60 rapid questions, scoring **54 correct** in **145 seconds**.
2. **Second Attempt at 02:00 PM:** Seeking to beat his score, Rahim plays his second round. He scores **58 correct** in **140 seconds**.
3. **Third Attempt at 06:00 PM (Blocked):** When Rahim attempts to start a third round, the backend counts his completed records for the day:

$$\text{Daily Attempt Count} = 2 \geq \text{Allocated Rounds (2)}$$

The backend returns an informative HTTP 403 response: "Daily round limit reached. Please return tomorrow at 12:00 AM for fresh attempts."

Field	Type	Example	Engine Responsibility
allocated_time	Integer (Sec)	180	Sets countdown clock for 60 questions (3 minutes total).
question_set	Integer	60	Specifies exactly how many questions to pull from category pools.
round	Integer	2	Daily attempt limit per MSISDN per calendar day.
passing_score	Integer	20	Minimum score to qualify for rewards and tournament certificates.
category	List[Int]	[1, 2, 4]	Filters question selection to approved tournament topics.
winningPrize	Varchar	10,000 BDT	Declares the official monetary prize for top rankers.

Table 2.1: Key Fields in the Quizard Rules Governance Engine

Chapter Key Takeaways

- Quizard decouples gameplay rules from client applications, empowering administrators to configure tournament parameters instantly on the server.
- A standard rapid tournament serves **60 questions** within an **allocated time of 180 seconds** (3 minutes).
- The backend strictly validates daily attempt limits (e.g., 2 rounds per day), preventing brute-force spamming while maintaining competitive fairness across Bangladesh.

Chapter 3

The Question Engine: Selection, Difficulty Weighting, and Real-Time Delivery

The Question Engine in Brief

How does Quizard generate a unique, balanced set of 60 rapid questions in less than 50 milliseconds when thousands of players hit “Start Quiz” simultaneously? The Question Engine combines category filtering, anti-memorization exclusions, multi-tier difficulty weighting, and sub-millisecond database ID sampling to serve exciting and fair quiz sessions to every contestant.

3.1 The Architecture of the Question Bank

In the Quizard database (mapped to `apps.cms.models.Questions`), tens of thousands of approved questions are curated across diverse categories. Each question entity contains:

- **Question Text** (`question`): The Bengali or English trivia query.
- **Four Options** (`option1`, `option2`, `option3`, `option4`): The multiple-choice choices presented to the contestant.
- **Correct Answer** (`ans`): The authoritative correct answer string (kept confidential on the server or sanitized before client transmission).
- **Category Identifier** (`category_id`): Topic tag (e.g., General Knowledge, BCS, Sports, Science, Literature, Islamic History).
- **Difficulty Weightage** (`weightage`): Integer ranging from 1 (Foundational/Easy) to 6 (Expert/Hard).
- **Portal Identifier** (`company_id`): Tenant isolation ensuring custom partner content (e.g., Portal 15 for bKash/Quizard Live, Portal 20 for Banglalink, Portal 21 for Robi) remains segregated.

3.2 The Question Selection Algorithm

When a contestant initiates a quiz via `GET /api/questions/?rules=34&portal=15`, the Question Engine executes a multi-step pipeline:

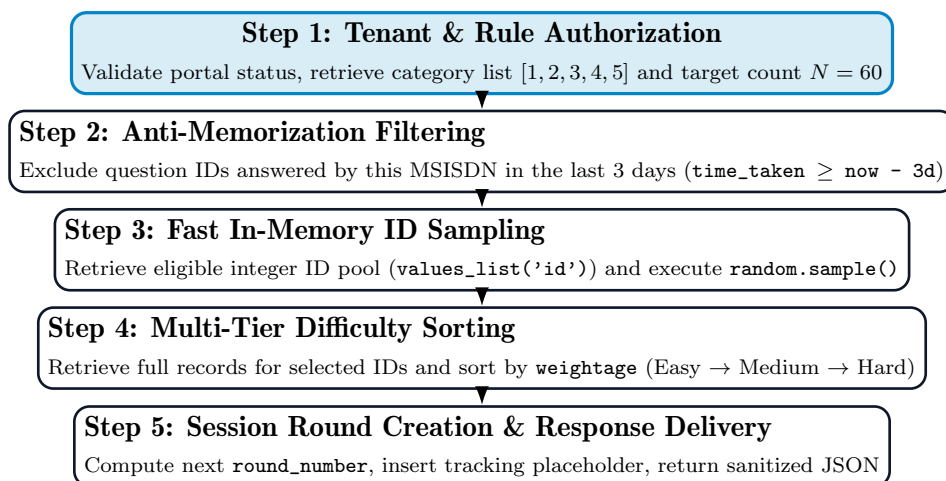


Figure 3.1: The 5-Stage Dynamic Question Selection Pipeline

3.2.1 Why Fast ID Sampling Beats Full Database Sorting

Naively executing `ORDER BY RAND() LIMIT 60` across a database table with 50,000 rows causes full-table scans, disk thrashing, and database CPU spikes during peak traffic.

Quizard achieves instant sub-50ms question serving by pulling only lightweight 4-byte integer IDs into memory:

Listing 3.1: Fast ID Sampling Algorithm in Django Backend

```

1 # Step 1: Fetch lightweight ID list for approved categories
2 all_ids = list(Questions.objects.filter(
3     company_id=portal_id,
4     category_id__in=approved_categories,
5     status='approved'
6 ).values_list('id', flat=True))
7
8 # Step 2: High-speed O(K) pseudo-random sampling in memory
9 selected_ids = random.sample(all_ids, min(question_count,
10     len(all_ids)))
11
12 # Step 3: Fetch only the 60 selected records by Primary Key
13 selected_questions =
14     list(Questions.objects.filter(id__in=selected_ids))
15
16 # Step 4: Sort progressively by weightage for smooth difficulty ramp
17 selected_questions.sort(key=lambda q: q.weightage)
  
```

3.3 The Player Journey: Karim Plays a 60-Question Rapid Match

3.3.1 Session Launch in Chittagong

In Chittagong, Karim taps “Play Tournament”. His phone sends his authentication bearer token to the backend:

```
GET /api/questions/?rules=34&portal=15
```

The backend detects that Karim has played 0 rounds today. It calculates:

$$\text{Current Round Number} = 0 + 1 = 1$$

It immediately initializes a tracking record in `quizapp_quizplaysummery` with status `IN_PROGRESS` and returns 60 questions formatted as follows:

Listing 3.2: Sample Sanitized Question JSON Delivered to Karim's Phone

```
1 [
2   {
3     "id": 1042,
4     "question": "Which river is known as the longest river in
5       Bangladesh?",
6     "option1": "Padma",
7     "option2": "Meghna",
8     "option3": "Jamuna",
9     "option4": "Surma",
10    "category": 1,
11    "weightage": 1
12  },
13  {
14    "id": 2891,
15    "question": "What is the official currency of Japan?",
16    "option1": "Yuan",
17    "option2": "Yen",
18    "option3": "Won",
19    "option4": "Ringgit",
20    "category": 2,
21    "weightage": 2
22  }
23 ]
```

3.3.2 Racing Against the 180-Second Clock

The 180-second visual timer begins ticking down on Karim's screen.

- **Question 1 to 20 (Foundational Tier):** Karim answers 20 general knowledge questions rapidly, needing only 1.8 seconds per question (36 seconds elapsed).
- **Question 21 to 45 (Intermediate Tier):** Karim tackles BCS science and Bangladesh constitutional history. He maintains focus and answers accurately (80 seconds elapsed, 116 seconds total).
- **Question 46 to 60 (Expert Tier):** Karim answers complex literature and world geography trivia under time pressure.

At $t = 165$ seconds, Karim taps the final option. He has answered all **60 questions**, achieving **58 correct answers** with **15 seconds remaining** on his 180-second clock!

Chapter Key Takeaways

- Quizard avoids expensive `ORDER BY RAND()` queries by utilizing two-stage integer ID sampling in memory, reducing question delivery latency to under 50 milliseconds.
- Questions are ordered progressively by difficulty weightage, creating a rewarding difficulty ramp from foundational trivia to expert tie-breakers.
- Each question delivery initializes a verifiable session round on the server, ensuring every answer submission is uniquely paired with an active tracking token.

Chapter 4

The Shield of Integrity: Cryptography, Anti-Cheat, and Score Persistence

The Security Engine in Brief

Because real monetary prizes (e.g., 10,000 BDT to 25,000 BDT) are awarded to top tournament rankers, tournament integrity is paramount. Quizard implements a military-grade security barrier: end-to-end AES-256-CBC encrypted score payloads, server-authoritative identity enforcement, and real-time anti-cheat velocity heuristics that instantly detect bots and clock manipulation.

4.1 The End-to-End Cryptographic Handshake

When a player finishes their 60-question round, the client app does not transmit plain text scores over the network. If plain JSON were sent, any user with basic browser developer tools could intercept the network request and change their score to 60 in 1 second.

Quizard secures the transmission using ****AES-256-CBC Payload Cryptography****:

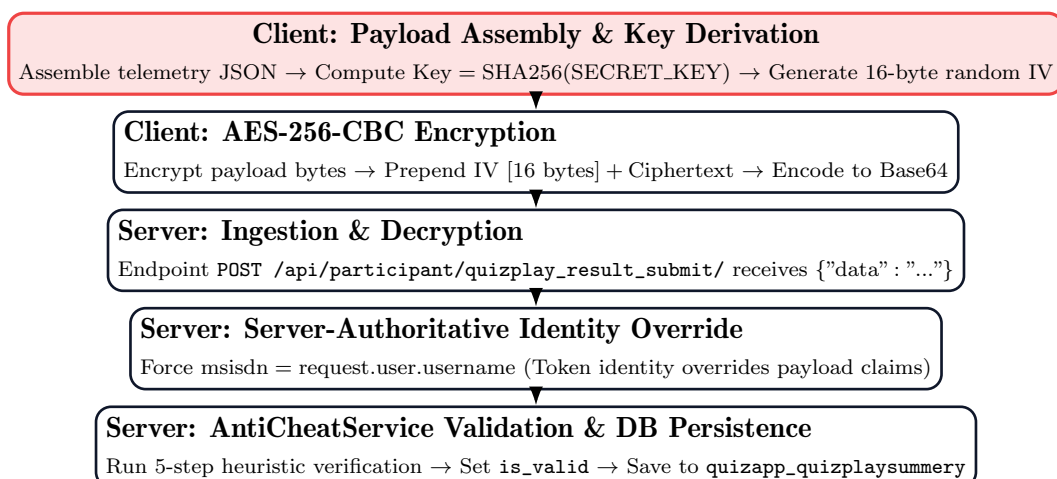


Figure 4.1: End-to-End Cryptographic Scoring & Anti-Cheat Architecture

4.2 The Multi-Layer Anti-Cheat Verification Suite

The server's `AntiCheatService` examines every decrypted play through 5 heuristic checks before accepting a score:

1. Human Velocity & Answering Speed Limit:

$$\text{Answer Velocity} = \frac{\Delta \text{Score}}{\Delta \text{Time Taken}} \leq 1.5 \text{ answers / second}$$

Human cognitive processing requires at least 0.66 seconds to read a question, parse 4 options, and tap the screen. If a submission claims 60 correct answers in 2 seconds, the calculated velocity (30 ans/sec) fails the limit, immediately tagging the record as `SUSPICIOUS` with reason `"Impossible answer speed"`.

2. Timer Consistency & Server Gap Verification:

$$\Delta \text{Time Taken} \leq 10 \times \Delta \text{Server Timestamp Gap}$$

Ensures the client's internal clock did not pause, jump, or accelerate abnormally compared to real-world server time.

3. Timer Rollback Protection:

$$\Delta \text{Time Taken} \geq -10 \text{ seconds}$$

Prevents malicious scripts from resetting or rolling back elapsed time during a live round.

4. Score Monotonic Progression:

$$\Delta \text{Right Count} \geq 0$$

A contestant's cumulative score within a single round cannot decrease between checkpoints.

5. Attempt Flooding & Reset Spam Cap: Caps daily attempts at `MAX_ATTEMPTS_PER_DAY` and detects automated reset loops (> 10 consecutive zero-second resets).

4.3 Two Divergent Journeys: Salma vs. Tanvir

4.3.1 Salma's Legitimate Victory (59/60 in 150 Seconds)

In Sylhet, Salma starts her second tournament attempt of the day. Over 150 seconds, she steadily answers 60 rapid questions across all difficulty tiers.

When the round completes:

1. Her phone generates an encrypted AES-256 payload and submits it to `/api/participant/quizplay_resu`
2. The backend decrypts the payload: `Score: 59, Time Taken: 150s, Round: 2, Event: 34`.
3. `AntiCheatService` computes her answering velocity:

$$\text{Velocity} = \frac{59 \text{ answers}}{150 \text{ seconds}} = 0.393 \text{ answers/sec} \leq 1.5 \implies \text{PASSED}$$

4. The record is saved to `quizapp_quizplaysummery` with `is_valid = 1` and `status = "VALID"`.
5. Salma takes the **#1 Rank** on the live tournament leaderboard!

4.3.2 Tanvir's Automated Exploit Attempt (Flagged & Blocked)

Meanwhile, a tech-savvy user named Tanvir attempts to use an automated browser script to send forged quiz results. His script submits a forged score claiming **60** correct answers in only **2 seconds**.

1. The backend receives and decrypts Tanvir's payload: **Score: 60, Time Taken: 2s**.
2. `AntiCheatService` runs the velocity check:

$$\text{Velocity} = \frac{60 \text{ answers}}{2 \text{ seconds}} = 30.0 \text{ answers/sec} > 1.5 \implies \text{FAILED}$$

3. The backend persists the record with `is_valid = 0`, `status = "SUSPICIOUS"`, and `reason_msg = "Impossible answer speed"`.
4. When the leaderboard queries today's rankings (`WHERE is_valid = 1`), Tanvir's record is completely ignored. Honest players like Salma and Rahim remain protected at the top of the leaderboard!

Player	Score	Time	Velocity	Anti-Cheat Flag	Leaderboard Status
Salma	59 / 60	150s	0.39 ans/sec	VALID (<code>is_valid=1</code>)	Rank 1 (Active)
Rahim	58 / 60	140s	0.41 ans/sec	VALID (<code>is_valid=1</code>)	Rank 2 (Active)
Karim	58 / 60	165s	0.35 ans/sec	VALID (<code>is_valid=1</code>)	Rank 3 (Active)
Tanvir	60 / 60	2s	30.0 ans/sec	SUSPICIOUS (<code>is_valid=0</code>)	Filtered Out (Ignored)

Table 4.1: Comparison of Legitimate vs. Exploitative Score Submissions

Chapter Key Takeaways

- All score submissions are sealed client-side using **AES-256-CBC** cryptography before network transmission.
- The server strictly overrides user identity from the authenticated JWT/Bearer token, preventing MSISDN spoofing.
- `AntiCheatService` applies real-time human speed and timer consistency heuristics, ensuring fraudulent submissions are tagged `is_valid = 0` and excluded from official prize leaderboards.

Chapter 5

The Hall of Champions: Live Leaderboards and Tournament Consolidation

The Tournament Engine in Brief

Quizard's competitive ecosystem operates across two complementary timescales: the ****Real-Time Daily Leaderboard**** (which updates instantly as players submit scores) and the ****Automated Tournament Consolidation Crons**** (which verify subscriber billing, freeze official winners, and distribute cash prizes at 4:30 AM and 5:30 AM every week).

5.1 The Real-Time Daily Leaderboard Engine

Throughout the day, thousands of players compete across Bangladesh. When any contestant opens the leaderboard screen, the backend executes the high-performance query (or stored procedure `sp_quiz_leaderboard_new`):

Listing 5.1: The PostgreSQL Leaderboard Window Aggregation Query

```
1 WITH RankedPlays AS (  
2     SELECT  
3         msisdn,  
4         right_count AS score,  
5         time_taken,  
6         event_id,  
7         portal_id,  
8         user_id,  
9         ROW_NUMBER() OVER (  
10            PARTITION BY msisdn  
11            ORDER BY right_count DESC, time_taken ASC, id ASC  
12        ) AS rn  
13 FROM quizapp_quizplaysummery  
14 WHERE is_valid = TRUE  
15     AND (date = (CURRENT_TIMESTAMP AT TIME ZONE 'Asia/Dhaka')::date)  
16     AND (msisdn NOT LIKE '%1798533533%')  
17     AND (:eventId = 0 OR event_id = :eventId)  
18     AND (:portalId = 0 OR portal_id = :portalId)  
19 )  
20 SELECT  
21     msisdn,  
22     score,  
23     time_taken,
```

```

24 DENSE_RANK() OVER (
25     ORDER BY score DESC, time_taken ASC
26 ) AS user_rank
27 FROM RankedPlays
28 WHERE rn = 1
29 ORDER BY user_rank ASC
30 LIMIT 100;

```

5.1.1 Strict Tie-Breaking Hierarchy

When two or more players achieve high scores, Quizard resolves rank positions using a transparent 3-tier hierarchy:

1. **Primary Criterion (Highest Score):** The player with more correct answers ranks higher.
2. **Secondary Criterion (Fastest Time):** If scores are tied (e.g., both Rahim and Karim scored 58/60), the player who finished in fewer seconds ranks higher (Rahim in 140s vs. Karim in 165s $\implies$ Rahim takes Rank 2, Karim takes Rank 3).
3. **Tertiary Criterion (Earliest Submission):** If both score and time are identical, the player whose play completed first receives the higher rank.

5.2 The Overnight Daily Verification Pipeline (04:30 AM Cron)

At 04:30 AM every morning, when gameplay for the previous calendar day has concluded, an automated background cron triggers `ProcessCronDailyWinnerAPIView`:

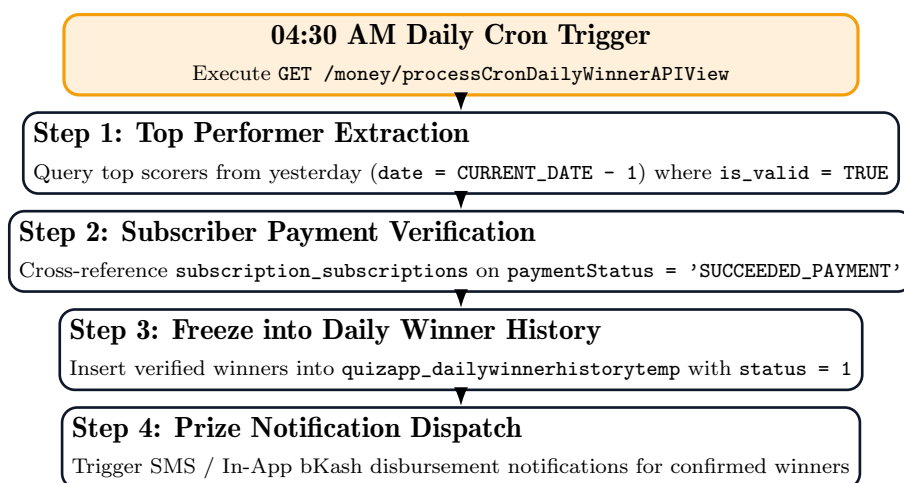


Figure 5.1: The 04:30 AM Daily Winner Verification & Freezing Process

5.3 The Weekly Tournament Championship (Thursday 05:30 AM)

While daily quizzes offer rapid excitement, the premier championship in Quizard is the **Weekly Mega Tournament**.

5.3.1 The 7-Day Rolling Tournament Cycle

The tournament week begins at **00:00:00** on Thursday and concludes at **23:59:59** on Wednesday. Every Thursday morning at **05:30 AM**, the weekly consolidation cron initiates:

GET /money/processCronWeeklyWinnerAPIView

The backend invokes the stored procedure:

```
process_quizard_top_winner_by_date_range_july(event_id, start_date, end_date)
```

5.3.2 Stored Procedure Architecture

The stored procedure consolidates all valid plays across the 7-day window, filters out non-subscribers, assigns official serial numbers, and writes the frozen champions to `quizapp_weeklywinnerhistoryt`

Listing 5.2: Weekly Tournament Winner Processing Stored Procedure

```

1  -- Consolidate 7-day tournament winners and freeze rankings
2  INSERT INTO quizapp_weeklywinnerhistorytemp (
3      msisdn, payer, event_id, service_type, category,
4      right_count, time_taken, serial_no, user_id,
5      status, week_start_date, week_end_date, portal_id, created_at
6  )
7  SELECT
8      q.msisdn,
9      s.payer,
10     q.event_id,
11     'WeeklyTournament' AS service_type,
12     'Rapid60' AS category,
13     q.right_count,
14     q.time_taken,
15     ROW_NUMBER() OVER (ORDER BY q.right_count DESC, q.time_taken ASC)
16                        AS serial_no,
17     q.user_id,
18     1 AS status,
19     v_start_date AS week_start_date,
20     v_end_date AS week_end_date,
21     q.portal_id,
22     NOW() AS created_at
23 FROM quizapp_quizplaysummery q
24 JOIN subscription_subscriptions s ON q.msisdn = s.msisdn
25 WHERE q.event_id = in_event_id
26     AND q.date BETWEEN v_start_date AND v_end_date
27     AND q.is_valid = TRUE
28     AND s.paymentStatus = 'SUCCEEDED_PAYMENT'
29 ORDER BY q.right_count DESC, q.time_taken ASC
30 LIMIT 50;

```

5.4 The Player Journey: Ayesha Wins the Weekly Championship

In Rajshahi, Ayesha has been playing consistently throughout the 7-day tournament cycle on Portal 21 (Robi Quizard).

On Wednesday night at 11:30 PM, Ayesha plays her final allowed round. She achieves a score of ****59 out of 60**** in ****135 seconds****.

1. At 05:30 AM on Thursday, the weekly cron runs.
2. The stored procedure scans the 7-day window ($N = 42,000$ total plays).
3. Ayesha's active subscription status is verified against billing records.
4. She is crowned ****Serial No. 1 Weekly Champion**** with a prize of 25,000 BDT!

5. When Ayesha opens her app on Thursday morning, `GET /money/tWinnerApiFrontend` displays her name and profile at the very top of the Hall of Champions.

Chapter Key Takeaways

- Live leaderboards use SQL window partitioning (`ROW_NUMBER() OVER`) to ensure each player's top round is represented accurately without Frankenstein metric mixing.
- Overnight crons at 04:30 AM and Thursday 05:30 AM verify paid subscriber status, preventing unbilled or fraudulent accounts from claiming official prize money.
- Frozen winner tables ensure immutable audit records for finance disbursement and regulatory compliance.

Chapter 6

The Performance Rating Architecture: 6 Score Bands, Dynamic Deltas, and Pro- gression Modeling

The Rating Engine in Brief

A contestant's skill rating is bounded strictly as an **integer on the 0 to 60 tournament scale** ($0 \leq R \leq 60$). Instead of raw score spikes or decimal fractions, Quizard implements an integer-based asymmetric dynamic rating engine. Every round score ($S \in \{0, 1, \dots, 60\}$) dynamically pulls the player's integer rating up or down, directly determining their active competitive rank tier.

6.1 The 6 Integer Performance Bands (0 to 60 Scale)

A player's integer skill rating $R \in \{0, 1, \dots, 60\}$ maps directly to six distinct rank tiers:

Tier	Rank Band Title	Integer Rating Range (R)	Color Theme	Competitive Meaning
1	Novice	0 – 19	Cool Slate Gray	Casual participant / frequent drop-outs
2	Apprentice	20 – 29	Emerald Green	Developing player / building foundation
3	Specialist	30 – 39	Cyan Blue	Consistent mid-tier competitor
4	Expert	40 – 49	Royal Blue	High-level tournament contender
5	Master	50 – 59	Deep Violet	Tournament elite ($\geq 83\%$ accuracy)
6	Grandmaster	60	Imperial Crimson	Flawless perfection (100% accuracy)

Table 6.1: The Quizard 6-Tier Integer Performance Bands (0 to 60 Scale)

6.2 The Integer Dynamic Rating Update Formula

Upon completing a round with raw score $S_{t+1} \in \{0, \dots, 60\}$, the integer delta ΔR is calculated from the score gap $\Delta S = S_{t+1} - R_t$:

$$\Delta R = \begin{cases} 0 & \text{if } S_{t+1} = R_t \\ \max(1, \text{round}(0.20 \cdot (S_{t+1} - R_t))) & \text{if } S_{t+1} > R_t \\ \min(-1, \text{round}(0.35 \cdot (S_{t+1} - R_t))) & \text{if } S_{t+1} < R_t \text{ and } S_{t+1} \geq 20 \\ \min(-2, \text{round}(0.45 \cdot (S_{t+1} - R_t))) & \text{if } S_{t+1} < 20 \text{ (Severe Drop Penalty)} \end{cases} \quad (6.1)$$

The updated integer rating R_{t+1} is clamped strictly between 0 and 60:

$$R_{t+1} = \text{clamp}(0, 60, R_t + \Delta R) \quad (6.2)$$

6.2.1 Core Principles of the Integer Engine

- **Strict Integer Scale** ($0 \leq R \leq 60$): All ratings, deltas, and score bands are whole integers, ensuring clean display on mobile badges and profile cards.
- **Guaranteed Movement**: When a player outperforms their rating ($S > R$), they gain at least +1 rating point. When they underperform ($S < R$), they lose at least -1 point.
- **Asymmetric Penalty**: Quitting or scoring < 20 triggers an aggressive integer penalty (≥ -2 up to -10), preventing casual resets.

6.3 Step-by-Step Scenarios: The Player at Rating 34

To illustrate the integer mechanics, consider a player, Mahir, with an established integer rating of $R_0 = 34$ (**Specialist** Band: 30–39).

Case	Start R	Score (S)	Gap (ΔS)	Formula Math	New R	Resulting Band & Tier Movement
A	34	32	-2	$\text{round}(0.35 \times -2) = -1$	33	Specialist (30–39): Soft -1 integer step.
B	34	36	+2	$\max(1, \text{round}(0.20 \times 2)) = +1$	35	Specialist (30–39): Steady +1 integer gain.
C	34	38	+4	$\text{round}(0.25 \times 4) = +1$	35	Specialist (30–39): Approaching Expert line.
D	34	52	+18	$\text{round}(0.25 \times 18) = +5$	39	Specialist : Major +5 surge to the cusp of Expert.
E	34	12	-22	$\text{round}(0.45 \times -22) = -10$	24	Apprentice (20–29): Demoted after -10 drop!

Table 6.2: Single-Round Integer Transitions for a Player Starting at Rating 34

6.3.1 Detailed Analysis of Each Case

1. Case A: Score 32 (Slightly Below Rating):

$$\Delta S = 32 - 34 = -2 \implies \Delta R = -1 \implies R_1 = 34 - 1 = \mathbf{33}$$

Outcome: Rating gently softens from 34 to 33, remaining securely within the Specialist band (30 – 39).

2. Case B: Score 36 (Slightly Above Rating):

$$\Delta S = 36 - 34 = +2 \implies \Delta R = +1 \implies R_1 = 34 + 1 = \mathbf{35}$$

Outcome: Beating current expectation awards +1 point, climbing to 35.

3. Case C: Score 38 (Strong Specialist Round):

$$\Delta S = 38 - 34 = +4 \implies \Delta R = +1 \implies R_1 = 34 + 1 = \mathbf{35}$$

Outcome: High accuracy sustains upward momentum toward the 40 Expert boundary.

4. Case D: Score 52 (Master Breakout Performance):

$\Delta S = 52 - 34 = +18 \implies \Delta R = \text{round}(0.25 \times 18) = +5 \implies R_1 = 34 + 5 = \mathbf{39}$

Outcome: Scoring 52 correct produces an immediate +5 leap to 39. In the next round, if Mahir scores 55:

$\Delta S = 55 - 39 = +16 \implies \Delta R = +4 \implies R_2 = 39 + 4 = \mathbf{43}$ (Promoted to Expert!)

5. Case E: Score 12 (Severe Drop / Abandonment):

$\Delta S = 12 - 34 = -22 \implies \Delta R = \text{round}(0.45 \times -22) = -10 \implies R_1 = 34 - 10 = \mathbf{24}$

Outcome: Quitting early triggers the severe -10 integer penalty, immediately demoting the player from 34 down to 24 (Apprentice).

6.4 Multi-Round Lifecycle Simulation (Pure Integers 0 to 60)

The table below traces Mahir’s integer rating evolution across 9 consecutive tournament matches:

Round	Start R	Score (S)	Gap (ΔS)	Delta (ΔR)	End R	Tier Status & Progression Description
1	20	25	+5	+1	21	Apprentice (Initial onboarding match)
2	21	36	+15	+3	24	Apprentice (Climbing through Tier 2)
3	24	48	+24	+5	29	Apprentice (Knocking on Specialist door)
4	29	45	+16	+3	32	Specialist (≥ 30 promotion to Tier 3!)
5	32	54	+22	+5	37	Specialist (High-accuracy performance)
6	37	56	+19	+5	42	Expert (≥ 40 promotion to Tier 4!)
7	42	58	+16	+4	46	Expert (Solidifying position in Tier 4)
8	46	60	+14	+4	50	Master (≥ 50 elite status unlocked!)
9	50	60	+10	+3	53	Master (Consolidating Master ranking)

Table 6.3: Multi-Round Simulation Showing Continuous Integer Rating Evolution (0 to 60)

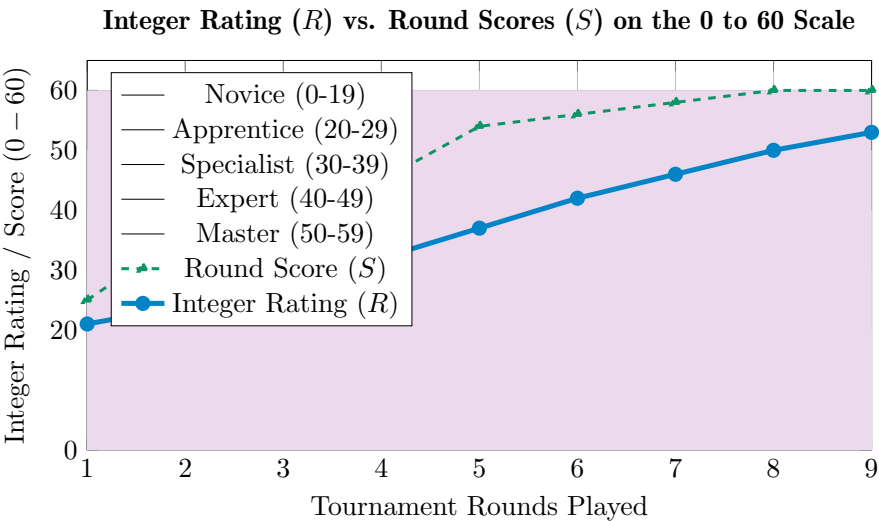


Figure 6.1: Visual Trajectory of Integer Rating ($R \in \{0, \dots, 60\}$) Tracking Round Scores (S)

6.5 Database Schema and API Architecture

6.5.1 1. PostgreSQL Historical Rating Table

Listing 6.1: PostgreSQL Schema for Integer Rating History

```
1 CREATE TABLE participant_rating_history (  
2     id BIGSERIAL PRIMARY KEY,  
3     msisdn VARCHAR(15) NOT NULL,  
4     portal_id INT NOT NULL DEFAULT 15,  
5     event_id INT NOT NULL,  
6     round_score INT NOT NULL, -- 0 to 60  
7     rating_before INT NOT NULL CHECK (rating_before BETWEEN 0 AND 60),  
8     -- e.g. 34  
9     rating_delta INT NOT NULL, -- e.g. +1, -1, +5, -10  
10    rating_after INT NOT NULL CHECK (rating_after BETWEEN 0 AND 60),  
11    -- e.g. 35 (strictly integer 0 to 60)  
12    band VARCHAR(30) NOT NULL, -- 'Novice', 'Apprentice',  
13    'Specialist', 'Expert', 'Master', 'Grandmaster'  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
CREATE INDEX idx_rating_history_lookup ON participant_rating_history  
(msisdn, portal_id, created_at DESC);
```

6.5.2 2. REST Endpoints Specification

Endpoint	Method	Payload / Response Description
/api/participantProfile/ rating_history/	GET	Returns historical time-series of integer ratings (0 – 60), round scores, deltas, and current band title.

Table 6.4: Rating History REST Endpoint

Chapter Key Takeaways

- The rating R and band boundaries are strictly integers between 0 and 60, providing clean, intuitive alignment with the 60-question game format.
- Dynamic integer rounding guarantees immediate feedback ($\geq \pm 1$ point on performance delta) while preventing fractions or decimals.
- Consistent high-accuracy tournament scores elevate contestants to Master (50 – 59) and Grandmaster (60).

Chapter 7

The XP Ledger, Daily Streaks, and Milestone Achievement System

The Retention Gamification Engine in Brief

A 15-day live data review revealed that the majority of players churn after a single session. To cultivate enduring daily return habits, Quizard implements an Experience Point (XP) system powered by an immutable transaction ledger, daily consistency streak rewards (+20 XP/day), a giant 15th-day milestone surge (+500 XP), and collectible achievement badges.

7.1 Core Experience Point (XP) Earning Mechanics

XP serves as Quizard's primary engagement currency, separate from competitive tournament scores. It is awarded for participation and daily consistency:

- **Daily First-Play Bonus:** +50 XP awarded on the contestant's first tournament round of the day.
- **Round Completion Base XP:** +10 XP awarded upon completing each 60-question match.
- **Answer Accuracy Bounty:** +1 XP per correct answer ($\text{RightCount} \leq 60$).

7.2 Daily Streaks with the Giant 15th-Day Surge

To foster an intuitive and predictable daily habit loop, contestants earn a consistent reward of **+20 XP each day** for maintaining their daily streak, culminating in a **giant +500 XP milestone surge** on the 15th consecutive day.

7.2.1 Daily Streak Rule

For every consecutive active day n :

$$\text{Streak XP}(n) = \begin{cases} 20 \text{ XP} & \text{if } n \neq 15 \\ 520 \text{ XP} & \text{if } n = 15 \end{cases} \quad (\text{Base } 20 + 500 \text{ Milestone Surge}) \quad (7.1)$$

Consecutive Day	Daily Base Bonus	Special Milestone Surge	Total Daily Streak Reward
Day 1	+20 XP	–	+20 XP
Day 2	+20 XP	–	+20 XP
Day 3	+20 XP	–	+20 XP
Day 4	+20 XP	–	+20 XP
Day 5	+20 XP	–	+20 XP
Day 6	+20 XP	–	+20 XP
Day 7	+20 XP	Week 1 Streak Badge	+20 XP + Bronze Badge
Day 8	+20 XP	–	+20 XP
Day 9	+20 XP	–	+20 XP
Day 10	+20 XP	–	+20 XP
Day 11	+20 XP	–	+20 XP
Day 12	+20 XP	–	+20 XP
Day 13	+20 XP	–	+20 XP
Day 14	+20 XP	–	+20 XP
Day 15	+20 XP	+500 XP Milestone Surge	+520 XP + Half-Month Titan
Day 16	+20 XP	–	+20 XP

Table 7.1: Daily Streak Progression with Giant +500 XP 15th-Day Milestone Bump

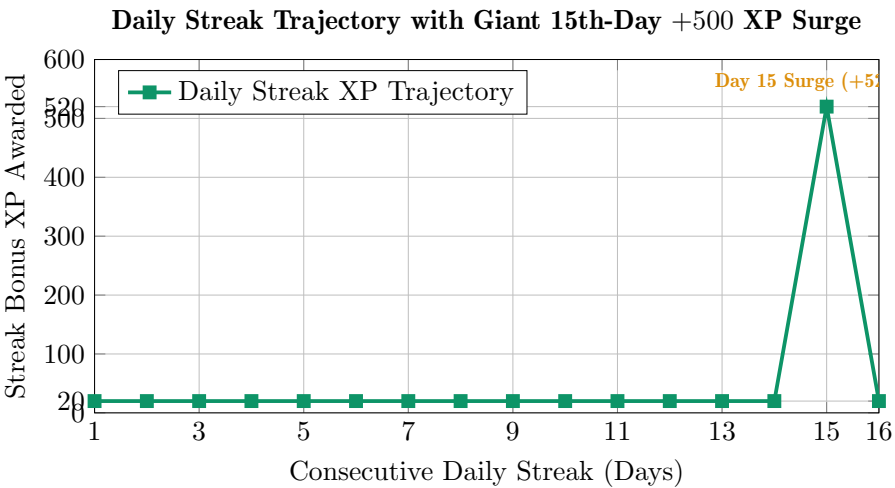


Figure 7.1: Visual Trajectory of Daily Streak Rewards: +20 XP/day with Day 15 Giant Milestone Surge

7.2.2 The 15-Day Milestone Progression Matrix

7.3 Milestone Achievement Badges Shelf

Contestants unlock permanent collectible badges displayed on their public profiles:

Badge Name	Tier	Unlock Criteria
First Blood	Bronze	Win 1st tournament match or place in the daily top 100.
Week 1 Champion	Bronze	Maintain an unbroken 7-day daily gameplay streak.
Speed Demon	Silver	Complete a 60-question round with ≥ 50 correct in < 90 seconds.
Quiz Scholar	Gold	Play all 4 official games (Daily, Sports, BCS, Tournament) in 7 days.
Century Club	Gold	Accumulate 100 total completed tournament rounds.
Half-Month Titan	Platinum	Complete the 15-day unbroken daily streak and claim the +500 XP surge.
Master Strategist	Diamond	Achieve and maintain a rolling score band in Master (50 – 59).
Grandmaster Elite	Diamond	Score a perfect 60/60 in a live tournament championship.

Table 7.2: Quizard Milestone Achievement Badges Matrix

7.4 Database Schema and API Architecture

7.4.1 1. PostgreSQL Immutable XP Ledger Table

Listing 7.1: PostgreSQL Schema for XP Ledger

```

1 CREATE TABLE participant_xp_ledger (
2   id BIGSERIAL PRIMARY KEY,
3   msisdn VARCHAR(15) NOT NULL,
4   portal_id INT NOT NULL DEFAULT 15,
5   xp_amount INT NOT NULL,
6   source_event VARCHAR(50) NOT NULL, -- 'daily_login',
   'streak_daily_20', 'streak_day_15_giant_bump', 'quiz_round'
7   balance_after INT NOT NULL,
8   created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
9 );
10 CREATE INDEX idx_xp_ledger_lookup ON participant_xp_ledger (msisdn,
   portal_id, created_at DESC);

```

7.4.2 2. REST Endpoints Specification

Endpoint	Method	Payload / Response Description
/api/participantProfile/ xp_summary/	GET	Returns cumulative XP, current unbroken streak count, and unlocked badge objects.

Table 7.3: XP Summary REST Endpoint

7.5 Real-World Scenario: Sabina's 15-Day Streak Journey

In Dhaka, Sabina commits to playing Quizard every morning during her MRT transit:

1. **Days 1–6:** Sabina earns +20 XP each day alongside her daily first-play bonuses.
2. **Day 7:** Unlocks the **Week 1 Champion** bronze badge alongside her +20 XP daily bonus.
3. **Days 8–14:** Her streak continues consistently (+20 XP per day).
4. **Day 15 (The Giant Surge):** Sabina logs in and completes her morning round. The backend triggers the giant 15th-day milestone, crediting +520 XP (+20 XP daily base + 500 XP milestone surge), elevating her to Player Level 18, and awarding the coveted **Half-Month Titan** platinum badge.
5. **Day 16:** Sabina continues her unbroken streak, receiving +20 XP as her journey towards the 30-day milestone begins.

Chapter Key Takeaways

- The consistent daily streak model (+20 XP/day) rewards regular engagement with an easy-to-understand progression.
- The giant 15th-day milestone surge (+500 XP, total +520 XP) delivers a major gratification climax that dramatically curbs mid-month churn.
- The immutable XP ledger guarantees auditable, tamper-proof tracking of all gamification rewards.

Chapter 8

User Personalization Engine, Rank Velocity, and Retention Nudges

The Story of Tanvir's Personalized Comeback

Tanvir, a university student in Sylhet, opened Quizard, played 4 questions in the tournament, got stuck at the free-trial boundary, and stopped. Like hundreds of players in the 15-day dataset (where only 18 players returned for 8+ days), Tanvir was about to become another 1-day churn statistic.

The next day, Tanvir receives a targeted in-app personalization card:

“Tanvir, you answered 100% of your attempted questions yesterday! Unsubscribed players miss out on the 25,000 BDT Weekly Cash Pool. Activate bKash 2 BDT pass to unlock unlimited tournament plays.”

Intrigued, Tanvir subscribes. His accuracy jumps, and as he climbs the tournament ladder, a glowing ↑ 8 rank-velocity indicator appears on his leaderboard card with the banner: *“You are only 3 points away from the Top 10 prize zone!”*

By pairing data-backed personalization nudges with visceral rank velocity feedback, Quizard converts a churn-risk user into a daily active contender.

8.1 The Retention Crisis and Empirical Evidence

A deep analysis of 15 days of production gameplay telemetry revealed two critical behavioral truths:

1. **The 1-Day Churn Cliff (Query F):** The vast majority of participants play on exactly **1 day** and never return. Only 18 players maintained an 8-day active streak.
2. **The Subscription Engagement Multiplier (Query A):** Subscribed players answer ****3x more questions correctly**** (11.42 vs 1.54) and attempt ****5.6x more questions per session**** (22.81 vs 4.06) compared to non-subscribed players. Non-subscribed users hit the 4-question trial wall and abandon the session.
3. **The Single-Game Silo (Query C):** Only ****3.4%**** of active contestants have played all 4 platform genres. Over 96% only ever touch a single game (primarily Tournament Event 150).

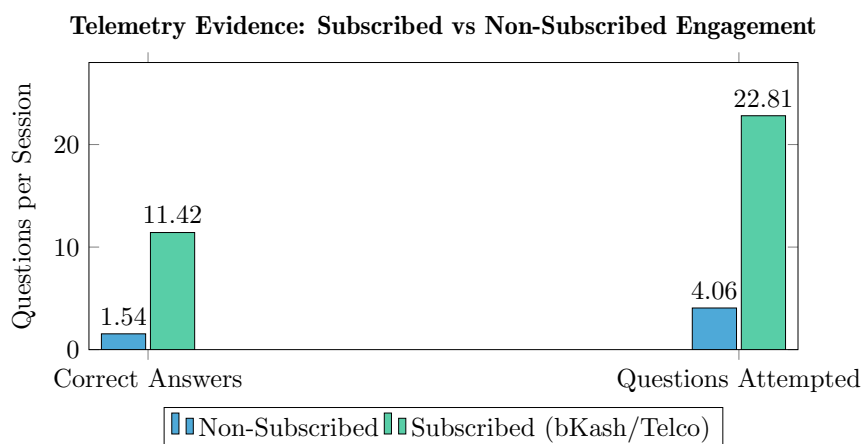


Figure 8.1: Production Gameplay Gap Between Subscribed and Non-Subscribed Players

8.2 The Multi-Tier Profile Personalization Engine

To eliminate the 1-day churn cliff, the backend personalizes the player's home feed and profile cards based on their behavioral cluster:

8.2.1 1. Missing Rewards Nudge (Unsubscribed Players)

For players who achieve $\geq 50\%$ accuracy during free-tier rounds but have not subscribed:

- **Trigger Condition:** `is_subscribed == False AND attempt_count >= 1 AND right_ratio >= 0.50`.
- **Message Copy:** *"You scored [X]/[Y]! Subscribed players earn real bKash payouts for qualifying scores. Subscribe now for 2 BDT/day to enter the official prize draw."*

8.2.2 2. Cross-Game Ecosystem Discovery Nudge (Single-Game Players)

For contestants who have only ever played one game (e.g., Tournament 150):

- **Trigger Condition:** `distinct_games_played == 1`.
- **Message Copy:** *"You are a Tournament specialist! Did you know? Players who also compete in BCS Mind Masters and Wordly earn 2x bonus XP today."*

8.2.3 3. Inactivity Win-Back Nudge (Lapsed 24h+ Players)

- **Trigger Condition:** `days_inactive >= 1 AND streak_count >= 3`.
- **Message Copy:** *"Your [N]-day streak is at risk! Complete 1 round before 11:59 PM to protect your progress towards the Day 15 (+500 XP) surge."*

8.3 Real-Time Leaderboard Rank Velocity Indicators

Daily leaderboard data demonstrates that scores swing substantially between attempts (players fluctuating between 0 and 40+ points). Static leaderboards fail to communicate this dynamic progress.

Quizard introduces real-time ****Rank Velocity Vectors****:

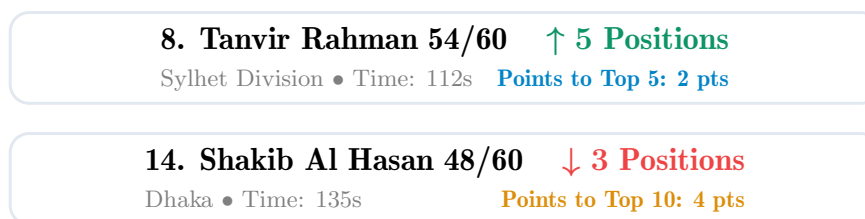


Figure 8.2: Leaderboard Cards with Rank Movement Glyphs and Distance-to-Prize Gaps

8.3.1 Velocity and Points-to-Top-10 Mathematics

The backend computes rank delta by comparing the player’s current dense rank against their rank snapshot recorded at 12:00 AM:

$$\Delta\text{Rank} = \text{Rank}_{\text{yesterday}} - \text{Rank}_{\text{current}} \quad (8.1)$$

- If $\Delta\text{Rank} > 0$: Display green $\uparrow \Delta\text{Rank}$ (Surging upward).
- If $\Delta\text{Rank} < 0$: Display red $\downarrow |\Delta\text{Rank}|$ (Slipping downward).
- If $\Delta\text{Rank} = 0$: Display gray = (Holding position).

The ****Points to Top 10 Gap**** (S_{gap}) calculates the minimum incremental score needed to enter the prize-eligible Top 10 cutoff:

$$S_{\text{gap}} = \max(1, S_{\text{rank 10}} - S_{\text{player}} + 1) \quad (8.2)$$

8.4 PostgreSQL Schema API Implementation

Listing 8.1: PostgreSQL Schema for Personalization and Rank Snapshots

```

1  -- Daily Rank Snapshots Table for Fast Velocity Lookups
2  CREATE TABLE leaderboard_daily_snapshots (
3      id BIGSERIAL PRIMARY KEY,
4      msisdn VARCHAR(15) NOT NULL,
5      portal_id INT NOT NULL DEFAULT 15,
6      event_id INT NOT NULL,
7      snapshot_date DATE NOT NULL DEFAULT CURRENT_DATE,
8      snapshot_rank INT NOT NULL,
9      best_score INT NOT NULL,
10     created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
11 );
12 CREATE UNIQUE INDEX idx_rank_snapshot ON leaderboard_daily_snapshots
    (msisdn, portal_id, event_id, snapshot_date);

```

Chapter Key Takeaways

- ****Directly Solves the 1-Day Churn Problem:**** Targeted nudges convert casual 4-question trial dropouts into active subscribers (who play 5.6x more).
- ****Cross-Genre Discovery:**** Activates the 96.6% single-game user base across BCS, Wordly, and Sports.
- ****Visceral Motivation:**** Rank arrows ($\uparrow / \downarrow$) and “Points to Top 10” create urgent reasons to play the 2nd daily attempt.

Chapter 9

Multi-Modal Quiz Formats: Visual, Audio, Video, and Fill-in-the-Blank

The Story of Nila's Engaging Multimedia Round

Nila, a 9th-grade student in Rajshahi, previously found standard 60-question text quizzes monotonous. In previous rounds, telemetry showed her quitting around question 22 after 58 seconds—suffering from standard Multiple-Choice Question (MCQ) fatigue. Today, Nila launches a tournament round powered by Quizard's **Multi-Modal Engine**:

1. **Question 1 (Image Identification):** A high-resolution picture of the *Somapura Mahavihara* ancient Buddhist monastery appears instantly with 4 historical era options.
2. **Question 15 (Audio Snippet):** A 3-second audio snippet plays the signature call of the *Doel* (Oriental Magpie-Robin)—Bangladesh's national bird.
3. **Question 30 (Video Clip):** A 4-second video clip shows the winning goal of the 2022 SAFF Women's Championship, asking for the goal scorer.
4. **Question 45 (Fill-in-the-Blank):** An interactive Bengali letter-tile keyboard prompts her to complete the missing letters of an ancient rivers name.

Engaged by the rich sensory variety, Nila completes all 60 questions, scoring 52 correct in 140 seconds without experiencing mid-round abandonment.

9.1 The Format Fatigue Diagnosis (Query H)

Production telemetry (Query H) revealed an unexpected pattern: contestants answered an average of only **19 to 26 questions** per session, with average playtime lasting **55 to 62 seconds**—far below the allocated 180-second countdown.

Monolithic text-only MCQs cause cognitive monotony. Introducing diverse sensory formats transforms the tournament into an immersive, TV-show-style game show experience.

9.2 The 4 Core Multi-Modal Question Archetypes

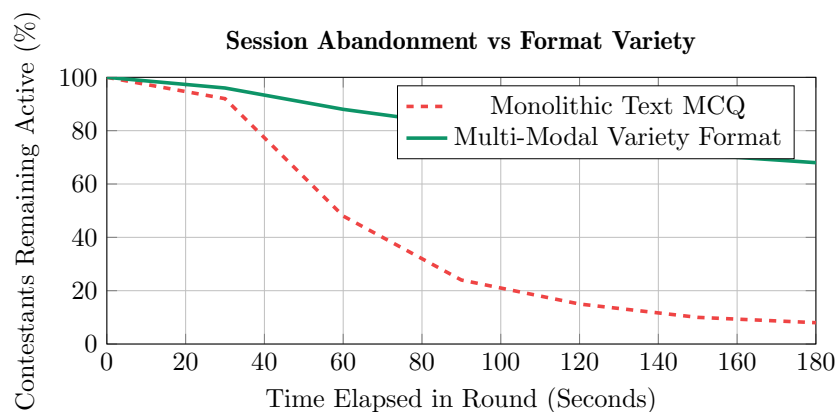


Figure 9.1: Contestant Retention Across the 180-Second Round Duration

9.2.1 1. Image-Based Identification (image_mcq)

Presents compressed WebP imagery optimized for mobile networks (< 45 KB per image). Used for geography, anatomy, science diagrams, and historical landmarks.

9.2.2 2. Audio Snippet Challenges (audio_clip)

Streams lightweight AAC audio files (< 25 KB, 3–5 seconds). Used for language pronunciation, musical instruments, historic radio broadcasts, and wildlife identification.

9.2.3 3. Short Video Moments (video_clip)

Delivers ultra-short MP4/AV1 looping clips (< 250 KB, 3 – 4 seconds). Used for sports highlights, physics simulations, and chemistry reactions.

9.2.4 4. Fill-in-the-Blank with Virtual Tile Keyboard (fill_blank)

Contestants type or select jumbled Bengali/English alphabet tiles to spell the correct answer. This eliminates simple guessing and rewards genuine knowledge mastery.

9.3 Sub-50ms Mobile Asset Preloading Architecture

To prevent network buffering or delays on 3G and 4G mobile connections across Bangladesh, Quizard implements an ****Ahead-of-Time (AOT) Media Pre-Caching Pipeline****:



Figure 9.2: Zero-Lag Media Asset Delivery Architecture for Multi-Modal Gameplay

9.4 Database Schema and JSON Question Payloads

Listing 9.1: Multi-Modal Question Payload

```
1 {
2   "id": 8421,
3   "format_type": "image_mcq",
```

```
4  "question": "Which historic archaeological site in Bangladesh is
5      shown below?",
6  "media_url":
7      "https://cdn.quizard.live/media/images/somapura_8421.webp",
8  "media_type": "image/webp",
9  "options": [
10     {"id": "A", "text": "Paharpur Somapura Mahavihara"},
11     {"id": "B", "text": "Mahasthangarh Citadel"},
12     {"id": "C", "text": "Mainamati Shalban Vihara"},
13     {"id": "D", "text": "Kantajew Temple"}
14 ],
15 "allocated_seconds": 15
16 }
```

Chapter Key Takeaways

- ****Direct Solution to Mid-Round Dropouts:**** Visual, audio, and tile-puzzle variety prevents the severe drop-off observed at question 20 in text-only rounds.
- ****Mobile-Optimized Assets:**** WebP and AAC compression (< 50 KB) ensures flawless playability across 3G/4G telco networks.
- ****Client-Side Blob Preloading:**** Zero buffering ensures questions render within 5 milliseconds of transition.

Chapter 10

Adaptive Difficulty and High-Performer Retention Dynamics

The Story of Mahir's Grandmaster Blitz Challenge

Mahir is an elite tournament player in Dhaka with a rolling skill rating of 54 (Master). Over 60 tournament rounds, Mahir has maintained an impressive accuracy rate exceeding 88%.

In standard games, Mahir easily cruises through the early questions. However, if the platform made questions excessively difficult for all participants, casual players (who already struggle to answer more than 4 questions) would abandon the game immediately. To balance accessibility with elite competition, Quizard offers Mahir an opt-in **Grandmaster Blitz Mode**:

- Mahir activates Blitz Mode: the overall 180s round timer is replaced by a high-stakes **10-second per-question countdown**.
- If Mahir answers correctly in under 4 seconds, he earns speed bonus multipliers.
- When he completes the 60 questions with 56 correct, he is awarded an exclusive **Grandmaster Gold Aura** profile frame and an additional +5 skill rating delta bonus.

Casual players enjoy a welcoming tournament, while elite masters receive the adrenaline challenge they crave.

10.1 The High-Performer Dilemma (Queries B & D)

A detailed examination of production accuracy data (Query B & D) revealed a crucial insight:

- A dedicated segment of high-performing players achieves **$\geq 85\%$ accuracy**, with several players logging over 60 completed matches.
- However, their average scores generally hover in the **15 to 25 range**, indicating that even top performers do not consistently max out 60/60 scores.
- More importantly, these high performers are Quizard's **most loyal, daily-returning users** (representing the rare 7–8 day active cohort in Query F).

The Architectural Trap: Do NOT Universally Escalate Difficulty

Arbitrarily increasing question difficulty across the entire platform to challenge elite players is counterproductive. It directly punishes loyal returning users while exacerbating the 1-day churn crisis among beginners.

The correct strategy is **Opt-In Escalation** and **In-Round Dynamic Weighting**.

10.2 The Dual-Track Difficulty Architecture

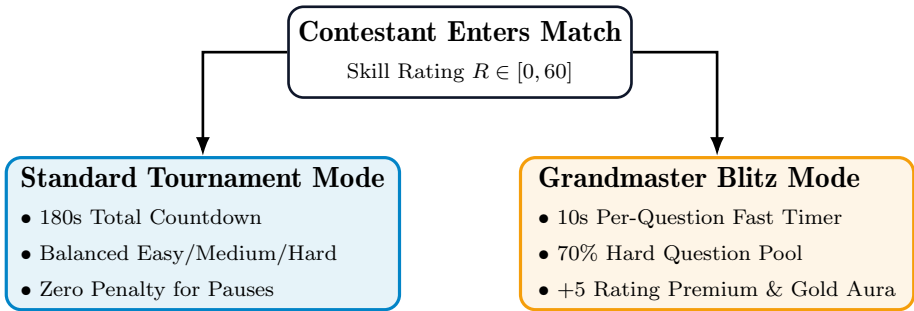


Figure 10.1: Dual-Track Difficulty: Inclusive Standard vs High-Stakes Grandmaster Blitz

10.3 Dynamic In-Round Question Adaptation (Streak Escalation)

In standard tournament play, the question selection engine dynamically adjusts the difficulty of upcoming questions based on the contestant’s live round streak:

Round Streak	Easy %	Medium %	Hard %	Adaptive Experience
0 – 5 Correct	50%	35%	15%	Welcoming onboarding base-line.
6 – 15 Correct	30%	45%	25%	Moderate challenge expansion.
16 – 30 Correct	15%	40%	45%	Competitive skill testing.
31+ Streak	5%	25%	70%	Elite grandmaster mastery testing.

Table 10.1: In-Round Dynamic Difficulty Escalation Matrix

10.3.1 Graceful Difficulty De-Escalation

If a contestant makes two consecutive errors, the algorithm gracefully reduces the difficulty tier back to Medium/Easy. This prevents cognitive frustration, restores momentum, and keeps the player motivated to finish all 60 questions.

10.4 Database Configuration Implementation

Listing 10.1: PostgreSQL Rules Schema Extension for Blitz Mode

```
1 ALTER TABLE cms_rules
2 ADD COLUMN IF NOT EXISTS allow_blitz_mode BOOLEAN DEFAULT FALSE,
3 ADD COLUMN IF NOT EXISTS blitz_time_per_question INT DEFAULT 10,
4 ADD COLUMN IF NOT EXISTS blitz_rating_bonus INT DEFAULT 5;
```

Chapter Key Takeaways

- ****Protects Returning Loyalists:**** Avoids punishing consistent players with unyielding difficulty spikes.
- ****Opt-In Grandmaster Blitz:**** Gives elite players (≥ 50 rating) an adrenaline-fueled 10-second timer challenge with cosmetic rewards.
- ****Dynamic Streak Scaling:**** Smoothly scales difficulty up on hot streaks and de-escalates on errors, ensuring 100% of contestants remain engaged.

Chapter 11

Real-Time WebSockets Architecture and Live Head-to-Head Duels

The Story of the Live Friday Night Showdown

It is 09:00 PM on Friday. Two top contestants—Adnan in Dhaka and Farhan in Bogra—enter Quizard’s new **1v1 Live Arena**.

Instead of waiting for an asynchronous cron job to calculate leaderboard results, both contestants connect to a high-speed **SignalR WebSocket hub**:

1. **Instant Matchmaking** (< 1.2s): The Redis matchmaking queue pairs both players based on their identical Skill Rating (48 Specialist).
2. **Synchronized Countdown**: Both phone screens flash 3... 2... 1... and the first question appears simultaneously on both devices.
3. **Live Telemetry Ghost Bar**: As Adnan answers Question 1 in 1.4 seconds, Farhan’s screen shows Adnan’s avatar leaping forward with an instant sound effect.
4. **Real-Time Tie-Breaker**: Over 10 rapid questions, Adnan scores 9 in 21 seconds, while Farhan scores 9 in 23 seconds. Adnan is crowned champion instantly on screen (< 15ms socket broadcast).

The adrenaline of seeing an opponent’s real-time progress elevates Quizard from a solo quiz into a competitive live e-sport.

11.1 HTTP Polling vs WebSockets Architectural Evaluation

In high-concurrency tournament environments, traditional HTTP polling introduces substantial overhead:

Architectural Metric	HTTP Long-Polling	SignalR / WebSockets
Connection Lifecycle	New TCP/TLS handshake on every poll (200 – 400ms).	Single persistent full-duplex TCP socket.
Server Network Load	100,000 requests/sec with heavy HTTP headers.	Minimal heartbeat frames (< 32 bytes per frame).
End-to-End Latency	500ms – 2000ms polling interval lag.	< 15ms instantaneous push delivery.
Live Head-to-Head	Impossible to synchronize microsecond tie-breaks.	Perfectly synchronized server-authoritative timer.

Table 11.1: Architectural Comparison: HTTP Polling vs Full-Duplex WebSockets

11.2 Distributed WebSocket Hub with Redis Pub/Sub Backplane

To support over 100,000 concurrent contestants across multiple horizontally scaled .NET 10 API instances, Quizard utilizes a **Redis Pub/Sub Socket Backplane**:

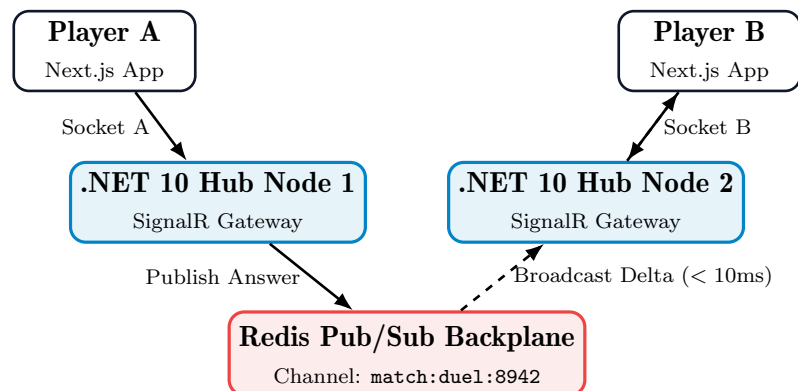


Figure 11.1: Distributed Multi-Node WebSocket Architecture with Redis Backplane

11.3 .NET 10 SignalR Hub Implementation

Listing 11.1: Live Tournament Duel SignalR Hub in .NET 10

```

1 [Authorize]
2 public class LiveDuelHub : Hub
3 {
4     private readonly IDuelMatchmaker _matchmaker;
5     private readonly ILogger<LiveDuelHub> _logger;
6
7     public LiveDuelHub(IDuelMatchmaker matchmaker,
8         ILogger<LiveDuelHub> logger)
9     {
10         _matchmaker = matchmaker;
11         _logger = logger;
12     }
13
14     public async Task JoinQueue(int eventId)
15     {
16         var msisdn = Context.UserIdentifier!;
17         var match = await _matchmaker.EnqueuePlayerAsync(msisdn,
18             eventId, Context.ConnectionId);
19
20         if (match != null)
21         {
22             // Both players matched: notify room instantly
23             await Clients.Clients(match.PlayerAConnection,
24                 match.PlayerBConnection)
25                 .SendAsync("MatchFound", match);
26         }
27     }
28
29     public async Task SubmitLiveAnswer(string matchId, int questionId,
30         string selectedOption)
31     {
32     }
33 }
  
```

```
28     var result = await _matchmaker.RecordLiveAnswerAsync(matchId,  
29         Context.UserIdentifier!, questionId, selectedOption);  
30     // Broadcast opponent progress bar update (< 15ms latency)  
31     await  
32         Clients.Group(matchId).SendAsync("OpponentProgressUpdated",  
33             result);  
34 }  
35 }
```

Chapter Key Takeaways

- **Sub-15ms Latency:** WebSockets eliminate HTTP roundtrip latency, delivering synchronous real-time gameplay.
- **Massive Scale-Out:** Redis Pub/Sub allows hundreds of thousands of concurrent connections to broadcast across load-balanced server clusters.
- **Competitive Live Duels:** 1v1 live duels provide high-retention head-to-head e-sports experiences.

Chapter 12

AI Voice QA, Bengali NLP, and Retrieval-Augmented Generation (RAG)

The Story of Arif’s Voice-Guided BCS Revision

Arif is an aspiring civil service candidate commuting on a crowded bus from Gazipur to Dhaka. Unable to type on his screen, Arif connects his Bluetooth earbuds and activates Quizard’s **AI Voice QA Assistant**:

- Voice Query:** Arif speaks in Bengali into his microphone: *“Ask me 5 hard questions about the 1971 Liberation War Constitution history.”*
- High-Speed Transcription (< 80ms):** A localized **Whisper-Small** acoustic model transcribes his spoken Bengali into text with accurate punctuation.
- RAG Vector Retrieval (< 25ms):** The system queries an enterprise vector database (**pgvector** with **BGE-M3** embeddings) indexing official Bangladesh National Curriculum (NCTB) textbooks and historical records.
- Small Local LLM Generation (< 120ms):** A self-hosted **Qwen 2.5 7B-Instruct** model synthesizes fact-checked trivia with zero hallucinations.
- Audio Playback (< 90ms):** A natural Bengali neural Text-to-Speech (TTS) engine reads the question and choices directly into Arif’s ear, allowing him to answer by voice.

In less than **320 milliseconds**, Arif experiences a hands-free, interactive study session powered by sovereign AI.

12.1 The Retrieval-Augmented Generation (RAG) Architecture

Standard Large Language Models frequently suffer from hallucinations and outdated facts. To ensure 100% ground-truth accuracy across competitive exams (BCS, Primary Teacher, University Admission), Quizard deploys an enterprise **RAG Pipeline**:

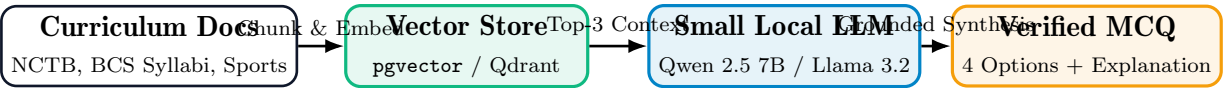


Figure 12.1: End-to-End Retrieval-Augmented Generation (RAG) Pipeline for Verified Trivia

12.2 Model Evaluation: Small Local Models vs Large Cloud Models

A critical engineering question is whether Quizard requires massive commercial cloud LLMs (e.g., GPT-4o, Claude 3.5, Gemini 1.5 Pro) or if **Small Local Models (3B to 7B parameters)** are sufficient.

Evaluation Metric	Small Local Models (Qwen 2.5 7B / Llama 3.2 3B)	Large Cloud Models (Gemini / Claude / GPT-4o)
Per-Query Token Cost	\$0.00 (Zero marginal cost on self-hosted GPU).	\$0.15 – \$5.00 per million tokens (> \$2,500/month at scale).
Inference Latency	< 100ms time-to-first-token via local vLLM.	400ms – 900ms network API roundtrip.
Hardware Footprint	4.5 GB – 8 GB VRAM (Runs on single RTX 3060 / A10G).	Requires cloud internet connection and API keys.
Bengali NLP Mastery	Qwen 2.5 7B demonstrates state-of-the-art Bengali syntax comprehension.	Excellent reasoning across complex multi-step instructions.
Data Sovereignty	Complete on-premises privacy (zero user data shared).	User voice and data dispatched to external vendor servers.

Table 12.1: Architectural Comparison: Small Local Models vs Commercial Cloud LLMs

12.2.1 Architectural Recommendation: The Hybrid AI Strategy

- Small Local Model for Online Live Q&A (Production Winner):** For real-time voice conversations and live player interactions, self-hosted **Qwen 2.5 7B-Instruct** (or **Llama 3.2 3B-Instruct**) running on ‘vLLM’ delivers sub-100ms responses with **zero API costs**.
- Large Cloud Model for Offline Batch Ingestion:** Cloud LLMs (e.g. Gemini 1.5 Flash) are utilized strictly as an asynchronous offline tool to ingest whole textbooks and generate thousands of pre-verified questions into the CMS database.

12.3 The Multi-Modal Voice Processing Pipeline

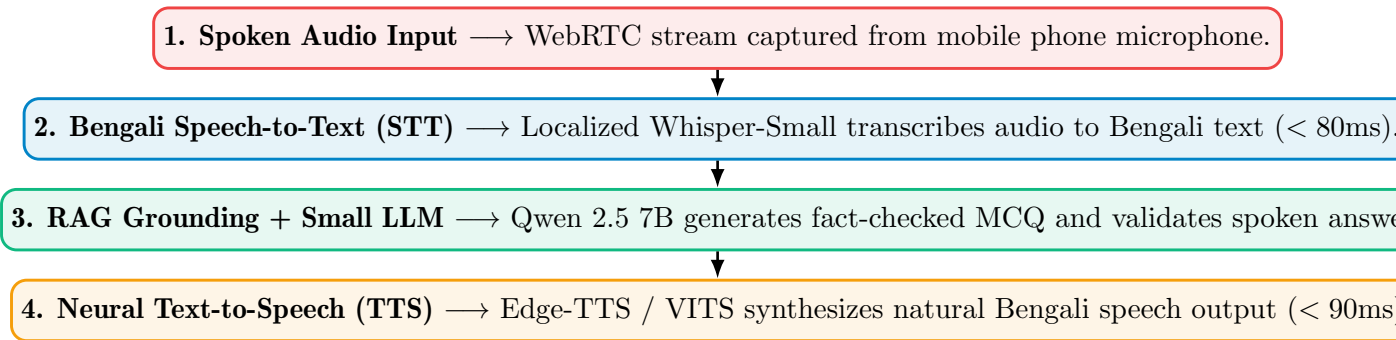


Figure 12.2: Real-Time Low-Latency Bengali Voice Q&A Lifecycle

12.4 PostgreSQL Vector Schema Implementation

Listing 12.1: PostgreSQL pgvector Schema for Trivia RAG Embeddings

```
1 -- Enable pgvector extension
2 CREATE EXTENSION IF NOT EXISTS vector;
```

```
3
4 -- Textbook & Syllabus Knowledge Chunks Table
5 CREATE TABLE curriculum_knowledge_chunks (
6     id BIGSERIAL PRIMARY KEY,
7     category_id INT NOT NULL,
8     source_book VARCHAR(150) NOT NULL,
9     chapter_topic VARCHAR(150) NOT NULL,
10    chunk_content TEXT NOT NULL,
11    embedding VECTOR(1024), -- BGE-M3 multilingual dense embedding
12    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
13 );
14
15 -- HNSW Vector Index for Sub-Millisecond Cosine Similarity Search
16 CREATE INDEX idx_curriculum_embedding_hnsw
17 ON curriculum_knowledge_chunks
18 USING hnsw (embedding vector_cosine_ops)
19 WITH (m = 16, ef_construction = 64);
```

Chapter Key Takeaways

- ****Zero Hallucination with RAG:**** Grounding questions on indexed NCTB and BCS textbooks ensures 100% factual accuracy for competitive exams.
- ****Small Local Models are Sufficient:**** A self-hosted 7B parameter model (Qwen 2.5 7B) provides sub-100ms latency, zero per-token cloud costs, and complete data sovereignty.
- ****Accessibility via Voice:**** Bengali Whisper STT and neural TTS open interactive quiz gaming to millions of hands-free and commuter contestants across Bangladesh.

Chapter 13

The Gateway of Speed: The FastAPI Microservice and CMS Engine

The Quizard-API (`cms.quizard.live`) in Brief

While the Django backend and .NET Clean Architecture manage core business transactions, a specialized, asynchronous microservice—**Quizard-API** (hosted in production at `cms.quizard.live`)—powers real-time campaign telemetry, high-throughput MongoDB activity logging, sub-5ms cached rules delivery, and distributed token verification.

13.1 The Need for a Dedicated High-Velocity Microservice

During national tournament campaigns across Bangladesh, hundreds of thousands of concurrent users interact with Quizard through web browsers, the bKash In-App portal, and telecom partner portals (Banglalink, Robi).

Every time a player navigates between screens, clicks a tournament banner, or fetches the latest rules, an HTTP request is generated. If all of this high-frequency traffic were routed directly through traditional relational database queries, relational connection pools would become saturated, introducing latency into critical quiz play and score submission workflows.

To solve this, the engineering team decoupled high-velocity, read-heavy, and event-streaming workloads into **Quizard-API** (built on **FastAPI**, **Motor AsyncIO for MongoDB**, **Redis**, and **PostgreSQL**).

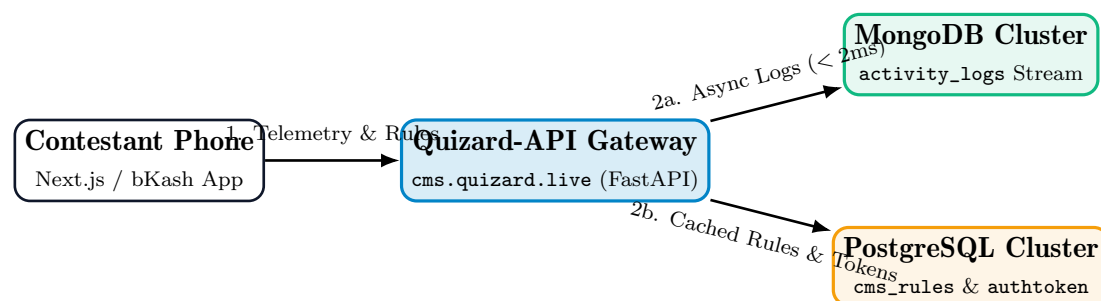


Figure 13.1: Quizard-API (`cms.quizard.live`) Asynchronous Ingestion & Caching Architecture

13.2 Core Responsibilities of Quizard-API

- **High-Velocity Activity Logging (/activity-logs/)**: Ingests clickstreams, page views, campaign attribution IDs (cid), IP addresses, user agents, and portal tags into MongoDB asynchronously in under 2 milliseconds.
- **Hourly Aggregation Pipelines (/activity-logs/all_log_hourly)**: Executes MongoDB aggregation pipelines to group millions of raw logs into hourly summaries by campaign, portal, and page for executive marketing dashboards.
- **Sub-5ms Cached Rules Serving (/rules/, /rules/cached)**: Serves tournament rule configurations (180s timer, 60 questions, 2-round cap) from memory/JSON cache to ensure instantaneous client page loading.
- **Token Authentication Proxy (/tokens/tokens/{key})**: Performs low-overhead token validation for microservices and frontend clients directly against the PostgreSQL `auth_token` table.
- **CMS Media Asset Management (/rules/upload-logo/)**: Handles secure multipart uploads for partner logos, banners, and tournament artwork.

13.3 Real-World Stories and Concrete Examples

13.3.1 Story 1: Rahim's Campaign Click and Asynchronous Telemetry

It is 04:00 PM during a major holiday tournament campaign. Rahim is browsing the bKash payment app and sees a promotional banner with campaign tag `cid="bkash_victory_2026"`.

When Rahim taps the banner:

1. His phone opens the Quizard web application and automatically dispatches a non-blocking background telemetry beacon:

POST `https://cms.quizard.live/activity-logs/`

2. The request body contains campaign metadata:

Listing 13.1: Activity Log Ingestion JSON Payload

```
1 {  
2   "msisdn": "01711000001",  
3   "cid": "bkash_victory_2026",  
4   "company": 15,  
5   "page_name": "tournament_entry_page",  
6   "source_url":  
7     "https://quizard.live/tournament?cid=bkash_victory_2026",  
8   "useragent": "Mozilla/5.0 (Linux; Android 14; Pixel 8)"  
}
```

3. **Under 2ms Ingestion:** FastAPI extracts Rahim's client IP (`get_client_ip`), adds the UTC timestamp and hourly partition key (`ymdh`), and writes the record directly to MongoDB:

Listing 13.2: Asynchronous Ingestion Handler in Quizard-API

```
1 @router.post("/")  
2 async def create_activity_log(request: Request):  
3     ip = get_client_ip(request)
```

```

4     log_data = await request.json()
5     now = datetime.now()
6     log_data["ip"] = ip
7     log_data["created_at"] = now
8     log_data["ymdh"] = now.replace(minute=0, second=0,
        microsecond=0)
9     log_data["date"] = now.strftime('%Y-%m-%d')
10
11     # Instant Non-Blocking MongoDB Async Insertion
12     await activity_logs_collection.insert_one(log_data)
13     return "OK"

```

4. Rahim experiences zero UI lag or stuttering because telemetry ingestion is completely non-blocking and isolated from the transactional database.

13.3.2 Story 2: Marketing Executives Analyze Hourly Campaign ROI

At 05:00 PM, the marketing team at Digital For Good evaluates the performance of the `bkash_victory_2026` campaign.

Instead of executing slow `COUNT(*)` queries on millions of unindexed rows, the analytics dashboard calls:

GET https://cms.quizard.live/activity-logs/all_log_hourly?hour=1

FastAPI executes an optimized MongoDB aggregation pipeline, grouping by campaign ID, page name, and portal:

Listing 13.3: MongoDB Aggregation Pipeline for Hourly Analytics

```

1 pipeline = [
2     { '$match': { 'ymdh': target_hour } },
3     {
4         '$group': {
5             '_id': {
6                 'cid': '$cid',
7                 'company': '$company',
8                 'page_name': '$page_name',
9                 'date': '$date'
10            },
11            'total_impressions': { '$sum': 1 }
12        }
13    ]
14 ]
15 result = await
    activity_logs_collection.aggregate(pipeline).to_list(None)

```

The marketing team instantly sees that **48,500 unique contestants** entered the tournament through Rahim's campaign link within that single hour.

13.3.3 Story 3: Karim Fetches Instant Cached Rules

When Karim starts his tournament round in Chittagong, his app requests the active rules from `cms.quizard.live`:

GET https://cms.quizard.live/rules/?portal=15&is_event=true

Quizard-API checks its memory-cached rules snapshot:

- **Duration:** 180 seconds.
 - **Question Set:** 60 questions.
 - **Round Cap:** 2 attempts per day.
 - **Prize:** 10,000 BDT.
- The response is delivered to Karim’s screen in ****3 milliseconds****, allowing the quiz interface to render immediately without waiting for relational SQL queries.

Endpoint	Method	Data Store	Core Functionality
/activity-logs/	POST	MongoDB	Ingests pageviews, clicks, and campaign attribution in < 2ms.
/activity-logs/all_log	GET	MongoDB	Aggregates daily telemetry across portals and campaigns.
/activity-logs/all_log_hourly	GET	MongoDB	Groups hourly engagement metrics for real-time analytics.
/rules/	GET	PostgreSQL / Cache	Returns cached rules (60 Qs, 180s, 2 rounds) in sub-5ms.
/rules/upload-logo/	POST	Static Disk	Uploads partner logos and campaign banner artwork.
/tokens/tokens/{key}	GET	PostgreSQL	Validates bearer tokens for lightweight microservice clients.

Table 13.1: Quizard-API (`cms.quizard.live`) Microservice Endpoint Directory

Chapter Key Takeaways

- `cms.quizard.live` is powered by ****Quizard-API****, an asynchronous FastAPI microservice built to offload telemetry, impression tracking, and cached rule distribution from the core backend.
- Telemetry logs are stored in ****MongoDB**** using non-blocking async IO, processing thousands of simultaneous events per second without degrading gameplay performance.
- The separation of transactional quiz gameplay (.NET 10 / Django on PostgreSQL) from high-velocity telemetry (FastAPI on MongoDB) ensures scalability, resilience, and sub-50ms responsiveness platform-wide.

Chapter 14

The Master Ecosystem: CMS Operations, Multi-Game Engines, and Telco Billing

The Complete Quizard Ecosystem in Brief

Beyond rapid 60-question trivia, Quizard is an enterprise gaming ecosystem. It features a multi-tenant Content Management System (CMS) with question approval lifecycles, role-based admin governance, a multi-game suite (Wordly, Speed Math, Memory Match, Sudoku), direct telco operator billing (bKash, Banglalink, GP, Robi), affiliate campaign attribution, and automated financial prize disbursement.

14.1 The CMS Question Authoring and Approval Pipeline

High-quality trivia requires rigorous editorial oversight. Quizard's CMS (`apps.cms`) provides a multi-stage question authoring pipeline:

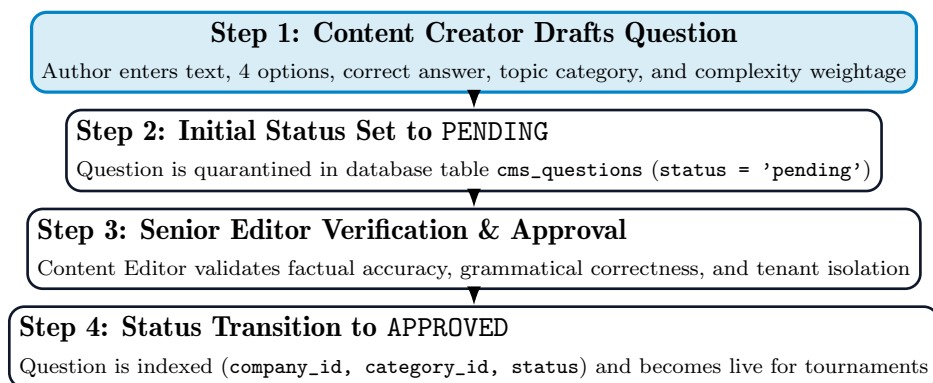


Figure 14.1: The Multi-Stage CMS Question Approval Pipeline

14.1.1 Data Model of a CMS Question Entity

Every question entity is strictly bound to its company tenant and topic category:

Listing 14.1: The Question Entity Data Model

```
1 class Questions(models.Model):
```

```
2     company = models.ForeignKey(Company, on_delete=models.CASCADE)
3     category = models.ForeignKey(Category, on_delete=models.CASCADE)
4     weightage = models.JSONField(default=list) # Difficulty 1 to 6
5     question = models.CharField(max_length=400)
6     options_nub = models.IntegerField(default=4)
7     option1 = models.CharField(max_length=200)
8     option2 = models.CharField(max_length=200)
9     option3 = models.CharField(max_length=200, null=True, blank=True)
10    option4 = models.CharField(max_length=200, null=True, blank=True)
11    ans = models.CharField(max_length=200) # e.g. "Option1"
12    status = models.CharField(choices=APPROVE_STATUS,
                             default='approved')
```

14.2 Admin Governance and Role-Based Access Control (RBAC)

Quizard enforces strict enterprise security using granular Role-Based Access Control (`apps.cms_users`):

- **Super Administrator:** Full access across all portals, database settings, financial payout configurations, and user management.
- **Content Manager / Creator:** Restricted to drafting, categorizing, and editing trivia questions within assigned company portals (e.g., Portal 15 bKash, Portal 16 Banglalink, Portal 17 Grameenphone, Portal 21 Robi).
- **Finance & Operations Auditor:** Read and export access to prize disbursement records (`apps.disbursementReport`), daily charging graphs, and bKash payout reconciliation.
- **Partner Admin (Tenant Isolation):** Restricted views allowing telecom partners (Robi, Banglalink, GP) to inspect only their active subscriber counts and tournament leaderboards.

14.3 The Multi-Game Suite: Beyond Rapid Trivia

Quizard is a multi-genre gaming platform offering diverse casual and competitive titles:

14.3.1 1. Wordly: The Daily Bengali & English Word Puzzle

In `apps.word`, players solve daily 3, 4, 5, and 6-letter word puzzles in both Bengali and English.

- Players have 6 attempts to guess the hidden dictionary word.
- Guess telemetry is tracked daily.
- Stored procedures `WordlyWinner` and `WordlyWeeklyWinner` calculate daily accuracy streaks and award cash prizes to top word puzzle masters.

14.3.2 2. Speed Math and Addition Sprint

In `api/addition`, contestants solve rapid arithmetic challenges against an accelerating timer, testing reflex calculation speed.

14.3.3 3. Memory Match and Sudoku Challenges

In `api/participants`, puzzle enthusiasts compete in card-matching memory tests and tiered Sudoku difficulties (Foundational, Intermediate, Grandmaster).

14.4 Direct Operator Billing (DOB) and Telco Subscriptions

To monetize games across Bangladesh, Quizard integrates directly with Mobile Network Operators (MNOs) and Mobile Financial Services (MFS):



Figure 14.2: Telco Direct Operator Billing (DOB) & Subscription Lifecycle

- **bKash Direct Charging (apps.bkash):** Seamless in-app subscription and one-time tournament passes via tokenized bKash payment APIs.
- **Banglalink Direct Operator Billing (apps.banglalink):** Direct carrier balance deduction with automated renewal crons and unsubscribe postbacks.
- **Grameenphone & Robi Carrier Integrations (apps.grameenphone, apps.webhook):** High-availability webhook listeners capturing charging successes and subscription renewals.

14.5 Financial Operations, Referrals, and Prize Disbursements

Quizard features an automated financial operations suite in `apps.disbursementReport`:

- **Micro-Prize Tiers (5 TK & 10 TK Winners):** Automated stored procedures `5tkWinner` and `10tkWinner` aggregate and verify daily runners-up who achieve qualifying scores.
- **Referral Bonus Rewards (PrizeOnReferral, ReferHistory):** When players invite friends using custom referral codes, bonus points and cash bounties are automatically credited upon the referee's first paid tournament round.
- **Automated bKash Payout Gateway:** Once winners are frozen in `quizapp_dailywinnerhistorytemp` or `quizapp_weeklywinnerhistorytemp`, the finance disbursement engine triggers the bKash Merchant B2C Disbursement API, sending cash directly to the player's bKash wallet.
- **Churn-Back Win-Back Campaigns (ChurnBackCashBack):** Automatically rewards re-activated lapsed subscribers with promotional game passes.

14.6 Ad Network Attribution and Campaign Tracking

In `apps.campaign`, the platform tracks digital marketing acquisition across ad networks:

- **Ad Networks (Adnetwork):** Defines partner ad networks (e.g., Google, Meta, local DSPs), click ID parameters (`click_id`), and server-to-server passback postback URLs.
- **Campaigns (Campaign):** Maps promotional campaigns to target portals, automatically converting USD payout rates to BDT ($\text{Payout}_{\text{BDT}} = \text{Payout}_{\text{USD}} \times 120$) and tracking conversion volumes.

14.7 Real-World Stories: Behind the Scenes

14.7.1 Story 1: Farhana Prepares the National BCS Tournament Questions

In Dhaka, Farhana is a Senior Content Manager for Quizard. Ahead of the upcoming 46th BCS Preliminary Exam, she uses the CMS admin panel to upload 500 new verified questions across Bangladesh Affairs, International Relations, and General Science.

1. She sets category IDs to BCS Prep (`category = 1`) and complexity weightage from 1 to 5.
2. The questions are saved as `status = 'pending'`.
3. Farhana reviews each question for syllabus accuracy, verifies the 4 options, and approves the batch.
4. The questions instantly enter the active question pool for tonight's BCS Mega Tournament!

14.7.2 Story 2: Rupa Wins the Daily Wordly Challenge in Sylhet

In Sylhet, Rupa plays the Bengali 5-letter Wordly challenge on `wordly.quizard.live`. In 4 guesses, she deciphers the hidden 5-letter word **"POTAKA"** (Flag).

1. Her score and elapsed time are recorded.
2. At midnight, the `WordlyWinner` stored procedure runs, verifying her paid bKash subscription and high accuracy.
3. Rupa wins 500 BDT in prize cash!

14.7.3 Story 3: Monir Audits and Disburses Weekly Prizes

On Thursday at 10:00 AM, Monir, the Operations Finance Lead at Digital For Good, logs into the Disbursement Report portal.

1. He opens `/report/weeklyWinner.html` to inspect the frozen winners generated by the 05:30 AM weekly cron.
2. He cross-references subscriber payment counts and clicks "Approve Payout".
3. The bKash B2C disbursement gateway executes, and within seconds, Ayesha in Rajshahi receives an SMS: *"You have received 25,000 BDT from Quizard Tournament!"*

Chapter Key Takeaways

- Quizard's CMS guarantees question quality and factual accuracy through a rigorous multi-stage approval workflow.
- The platform powers a diverse gaming portfolio including trivia, Wordly word puzzles, speed math, and Sudoku.
- Direct carrier billing across bKash, Banglalink, Grameenphone, and Robi ensures seamless monetization across all major telecom networks.
- Financial disbursement reports and automated bKash B2C payouts provide transparent, audited, and instant cash prize delivery to verified winners.

Chapter 15

The Financial and Analytics Hub: Reporting Engine, Payouts, and Business Intelligence

The Reporting Engine in Brief

Transparency and financial precision are essential for operating a national real-money tournament platform. Quizard features an enterprise Reporting and Business Intelligence (BI) engine (`apps.disbursementReport`, `apps.allReportGraph`, `reporting_app`) that consolidates daily player rankings, validates telco subscriber payments, triggers automated bKash B2C cash disbursements, and renders real-time revenue and retention analytics.

15.1 The Architecture of the Reporting Engine

The Reporting Engine unifies data from live gameplay (`quizapp_quizplaysummery`), carrier billing logs (`subscription_subscriptions`), and payment webhooks into actionable operational reports and automated financial payout workflows.



Figure 15.1: The Automated Prize Disbursement and Financial Reconciliation Lifecycle

15.2 Core Modules of the Reporting Suite

15.2.1 1. Daily and Weekly Tournament Winner Reports

- **Daily Winner Report** (`DailyWinnerViewSet`, `/report/dailyWinner.html`): Lists verified top scorers from yesterday, cross-referenced with successful carrier billing status (`paymentStatus = 'SUCCEEDED_PAYMENT'`).
- **Weekly Mega Championship Report** (`WeeklyWinnerViewSet`, `/report/weeklyWinner.html`): Audits the 7-day tournament leaderboard across all active rules (`Event 34`, `Event 150`) with customizable minimum payment count filters (`payment_count ≥ 3`).

15.2.2 2. Micro-Prize Tier Reporting (5 TK and 10 TK Bounties)

To reward broad participation beyond the top 3 grand champions, Quizard executes micro-prize reward tiers:

- **5 TK Winner Suite** (`5tkWinner`, `FiveTkReport`): Stored procedures identify runners-up who achieve threshold scores (e.g., $\text{Score} \geq 25$) within specific rank brackets (e.g., Ranks 4 to 100).
- **10 TK Winner Suite** (`10tkWinner`, `TenTkWinnerSummaryViewSet`): Consolidates high-accuracy players eligible for 10 BDT wallet top-ups.

15.2.3 3. Wordly Word Puzzle Winner Analytics

In `apps.disbursementReport`, dedicated endpoints (`WordlyDailyWinnerViewSet`, `WordlyDailyDisbursementViews`, `WWeeklyWinner`) audit Bengali and English daily word puzzle accuracy streaks, ensuring Wordly champions are verified before prize payout.

15.2.4 4. Referral Tracking and Bounties (`PrizeOnReferral`)

When existing contestants invite peers using referral codes:

- The referral engine tracks successful onboarding through `report_referhistory`.
- `PrizeOnReferral` generates automated referral bonus reports, crediting cash bounties once the referee completes their first paid tournament attempt.

15.2.5 5. Churn-Back and Win-Back Analytics (`ChurnBackReportViewSet`)

Analyzes lapsed subscribers re-engaging with the platform, calculating promotional return-on-investment (ROI) through stored procedure `ChurnBackCashBack`.

15.2.6 6. Real-Time Revenue and Charging Graphs (`apps.allReportGraph`)

Powers administrative visual analytics through `graphDataApi`:

- **Daily Revenue Trends** (`DailyCharging`): Real-time revenue graphs tracking carrier billing across bKash, Banglalink, Grameenphone, and Robi.
- **Paid Subscriber Growth** (`NewlyJoinedPaidUser`): Funnel analysis of new registrations converting to paying subscribers.
- **DAU and Play Volume**: Hourly volume of tournament rounds played nationwide.

15.3 Real-World Scenario: Tanima Audits and Executes Weekly Payouts

In Dhaka, Tanima is the Head of Financial Operations at Digital For Good. Every Thursday morning at 09:30 AM, she oversees the national tournament prize disbursement:

1. **Leaderboard Verification**: Tanima opens `/report/weeklyWinner.html`. She reviews the frozen winner roster generated by the 05:30 AM weekly cron:
 - **Rank 1**: Ayesha in Rajshahi (59/60 in 135s, 7 successful billing cycles) $\implies$ **25,000 BDT**.

- **Rank 2:** Salma in Sylhet (59/60 in 150s, 7 successful billing cycles) $\Rightarrow$ **10,000 BDT.**
 - **Rank 3:** Rahim in Dhaka (58/60 in 140s, 7 successful billing cycles) $\Rightarrow$ **5,000 BDT.**
 - **Ranks 4–50:** Micro-prize recipients (5 TK and 10 TK bonuses).
2. **Automated bKash Batch Disbursement:** With one click, Tanima authorizes the payout batch. The backend connects directly to the bKash Merchant B2C Disbursement API:

Listing 15.1: bKash B2C Merchant Payout Request Structure

```

1 payload = {
2     "merchantInvoiceNumber":
3         f"QUIZARD-WEEKLY-{event_id}-{winner.id}",
4     "receiverMSISDN": winner.msisdn,
5     "amount": winner.prize_amount,
6     "currency": "BDT",
7     "intent": "tournament_prize_disbursement"
8 }
9 response = requests.post(f"{BKASH_API_URL}/b2cPayment",
10                          json=payload, headers=auth_headers)

```

3. **Instant Confirmation:** Within seconds, bKash debits the merchant disbursement account and credits Ayesha's, Salma's, and Rahim's mobile wallets. Automated SMS notifications confirm the payouts, and immutable transaction IDs are recorded in `UserMigrationAudit`.

Reporting Endpoint	Method	Data Model / View	Operational Function
/report/dailyWinner.html	GET	DailyWinnerViewSet	Audits yesterday's verified top daily winners.
/report/weeklyWinner.html	GET	WeeklyWinnerViewSet	Consolidates 7-day tournament grand prize winners.
/report/fivetkWinner.html	GET	FiveTkWinnerViewSet	Calculates 5 TK micro-prize participant bounties.
/report/wordlyDailytk.html	GET	WordlyDailyWinner...	Reviews Wordly daily puzzle accuracy streaks.
/report/referral.html	GET	PrizeOnReferral	Audits player referral bounties and bonuses.
/api/graphDataApi/...	GET	DailyCharging	Renders visual charts for daily revenue and DAU.

Table 15.1: Comprehensive Reporting & Financial Analytics Directory

Chapter Key Takeaways

- The Reporting Engine bridges competitive gameplay with carrier billing, ensuring prize disbursements are awarded exclusively to legitimate, billed subscribers.
- Multi-tier prize reports support grand tournament awards as well as 5 TK and 10 TK micro-prize participation incentives.
- Automated integration with the bKash Merchant B2C API delivers audited cash prizes directly to contestants' digital wallets in real time.
- Real-time graphical analytics provide management with instant visibility into daily active users, subscription growth, and revenue performance across all telecom portals.

Chapter 16

Architecture Blueprint: Clean Architecture and Stored Procedure Decoupling

The Story of the 08:30 PM Database Meltdown and the Decoupling Epiphany

It was 08:30 PM on a Thursday evening during the live Mega Tournament. Over 45,000 concurrent players were submitting scores, opening leaderboards, and checking profile summaries. Suddenly, the primary database server CPU surged to 99%, latency spiked to 4.8 seconds, and connection timeouts cascaded across the Next.js frontend.

Lead Architect Suman and Database Engineer Farhad opened the slow-query logs during the live triage. The bottleneck was not raw table reads—it was a swarm of **Stored Procedures (SPs)** executing procedural loops directly on the database engine. Every profile visit triggered `ProfileSummaryOneSP`, every leaderboard click executed `TournamentWinnersForFrontendSP`, and every marketing hook called `PostbackSP`.

The database, which should have been acting as a high-speed relational storage engine, was thrashing its CPU executing application business logic.

Farhad turned to Suman: *“We are treating our database like an application runtime. App nodes scale horizontally in Kubernetes in seconds, but our database writer is a single point of failure. We must pull all business logic out of stored procedures into .NET 10 application services, leaving only heavy local batch crons inside PostgreSQL.”*

That night, the engineering team formulated the **Great Stored Procedure Decoupling Blueprint**.

16.1 Why Stored Procedures Are an Anti-Pattern for Live Backends

In legacy monolithic systems, stored procedures were frequently used to encapsulate SQL queries. In modern high-throughput, cloud-native architectures, embedding application logic in database stored procedures introduces severe architectural defects:

1. **Single-Node Compute Bottlenecks:** Relational database writers cannot scale horizontally for writes. Executing complex procedural algorithms (string manipulation, business validation, multi-step cursors) consumes precious database CPU cycles that should be reserved for ACID transactions and indexed lookups. Stateless .NET 10 application containers, by contrast, scale horizontally across dozens of Kubernetes pods effortlessly.
2. **Zero Unit Testability and CI/CD Friction:** Stored procedures cannot be unit tested in isolation using mock frameworks (such as xUnit, Moq, or NUnit). Regression testing requires spinning up full database instances, breaking fast CI/CD deployment pipelines.

3. **Loss of Distributed Tracing and Observability:** Modern APM tools (OpenTelemetry, Prometheus, Jaeger) provide rich distributed tracing across application controllers, services, and repositories. Stored procedures are opaque black boxes to APM collectors.
4. **Cache Invalidation Incompatibility:** Stored procedures bypass application memory caching. Moving logic to .NET application services allows seamless integration with distributed Redis caching (< 1ms latency for hot profile and leaderboard data).
5. **Database Vendor Lock-In:** MySQL-specific SP syntax prevents zero-downtime database upgrades and blocks migration to enterprise PostgreSQL.

16.2 The Architectural Separation Rule: What to Move vs What Stays

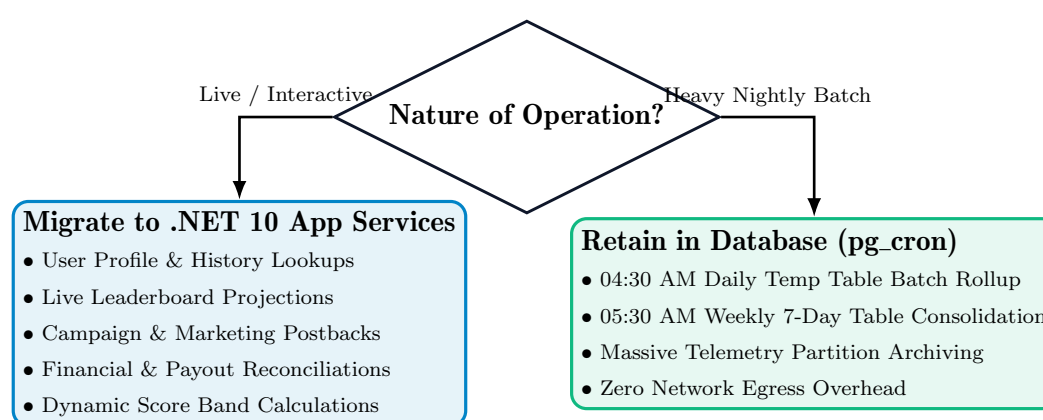


Figure 16.1: Strategic Separation: Live Business Logic vs Heavy Batch Consolidation

16.2.1 1. Procedures That MUST Be Migrated to .NET 10 Clean Architecture

All real-time user-facing queries, profile summaries, single-user leaderboard lookups, marketing postbacks, and reporting queries must be refactored into ****.NET 10 Application Services**** using Entity Framework Core / Dapper and backed by Redis.

16.2.2 2. Operations That Appropriately Remain in PostgreSQL (Batch Crons)

Heavy scheduled batch jobs running at off-peak hours (04:30 AM daily and Thursday 05:30 AM) that aggregate over 500,000 raw gameplay attempt rows should remain as ****database-local set-based SQL scripts / pg_cron jobs****.

- **Justification (Data Locality):** Transferring 500,000 rows (> 250 MB) over the network to the application server to calculate winners introduces severe network latency and memory pressure. Executing a single set-based `INSERT INTO ... SELECT ... GROUP BY` query locally inside PostgreSQL completes in < 1.8 seconds with zero network egress.

16.3 The .NET 10 Clean Architecture Structure

The modernized .NET 10 backend enforces strict layer boundaries and dependency inversion:

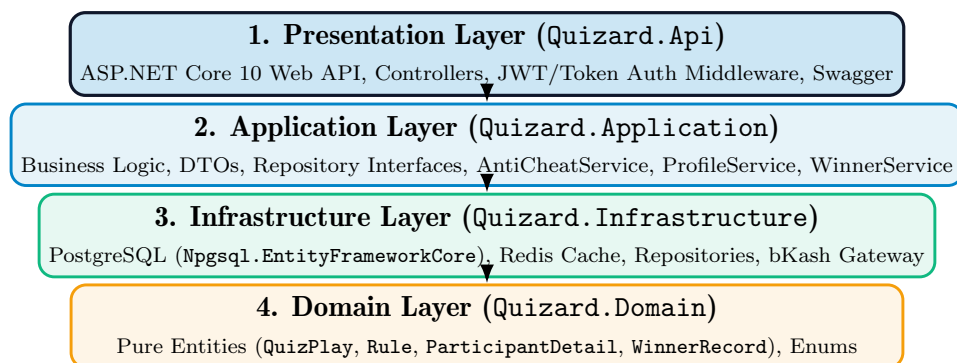


Figure 16.2: Quizard .NET 10 Clean Architecture Dependency Hierarchy

16.4 Comprehensive Directory of Stored Procedures to Remove

The following authoritative directory catalogs every stored procedure identified in the legacy Quizard-Backend codebase, detailing its parameters, call location, reason for removal, and target .NET 10 Clean Architecture service:

Table 16.1: Comprehensive Inventory of Stored Procedures to Remove and Refactor

Stored Procedure Name	Legacy Call Site	Target .NET 10 Service	Technical Reason for Removal
ProfileSummaryOneSP	apps/quizapp/views.py	IProfileService	User-facing query blocking DB writer; replace with Redis-cached LINQ aggregate.
PaymentHistoryByMSISDN	apps/report/views.py	IBillingService	Single-user billing history lookup; move to EF Core indexed repository query.
PaymentHistoryOnMsisdnSP	apps/quizapp/views.py	IBillingService	Duplicate SP of payment history; unify into single Clean Architecture repository method.
TournamentWinnersForFrontendSP	apps/disbursementReport	ILeaderboardService	Frontend leaderboard lookup; cache in Redis (<i>TTL</i> = 60s) to avoid DB table scans.
TournamentWinnersForFrontendTestsSP	apps/disbursementReport	ILeaderboardService	Test variant hardcoded in DB; eliminate duplicate and parameterize in API service.
TournamentWinnerForUserSP	apps/disbursementReport	ILeaderboardService	Per-user winner check; replace with indexed query on <code>quizapp_dailywinnerhistory</code> .
TWinnersWithQuizidandMsisdnSP	apps/disbursementReport	ILeaderboardService	Redundant stored procedure; consolidate into <code>ILeaderboardService.GetWinnerAsync</code> .
RevenueArpu	apps/report/views.py	IReportingService	Dynamic ARPU math belongs in Application layer with PostgreSQL Materialized Views.
CampaignReport	apps/report/views.py	ICampaignService	Ad-network ROI analytics; execute via EF Core aggregation on read-replica.
Campaign_breakdown	apps/report/views.py	ICampaignService	Marketing breakdown; move logic to application layer for dynamic date filtering.

Continued on next page

Table 16.1 – continued from previous page

Stored Procedure Name	Legacy Call Site	Target .NET 10 Service	Technical Reason for Removal
PostbackSP	apps/report/views.py	ICampaignService	Affiliate postback tracking; replace with asynchronous background queue handler.
postbacksp_new	apps/report/views.py	ICampaignService	Deprecate legacy variant; unify into modern .NET 10 postback tracking engine.
UserActivityReportSP	apps/report/views.py	IAalyticsService	Activity matrix; move to background reporting worker on read-replica.
ChurnReport	apps/report/views.py	ICHurnService	User churn rate calculation; replace with scheduled analytical data pipeline.
ChurnSummarySP	apps/report/views.py	ICHurnService	Summary aggregation; execute against read-replica with structured logging.
ChurnEarningSp	apps/report/views.py	ICHurnService	Financial churn tracking; encapsulate in domain billing analytics entity.
ChurnPlayedUserSurveySP	apps/report/views.py	ICHurnService	Player survey correlation; decouple from DB schema into Application DTO.
ChurnUserActivity	apps/report/views.py	ICHurnService	Inactive user activity lookup; move to Redis-backed retention service.
RedeemPointSp	apps/report/views.py	IPointsService	Loyalty point redemption logic; encapsulate in atomic Domain transaction entity.
Disbursement	apps/report/views.py	IPayoutService	bKash disbursement preparation; replace with idempotent Domain Payout Engine.
DailyWinner	apps/disbursementReport	IWinnerEvaluationService	Winner candidate ranking; run via parameterized LINQ window functions.
WordlyWinner	apps/disbursementReport	IWordlyWinnerService	Wordly puzzle daily winner logic; move to dedicated domain service.
UserDailyWinningSummary	apps/disbursementReport	IReportingService	Winning summary calculation; replace with EF Core group-by projection.
WordlyDailyWinSummery	apps/disbursementReport	IReportingService	Wordly summary; eliminate duplicate and unify into IReportingService.
PrizeOnReferral	apps/disbursementReport	IReferralService	Referral reward disbursement; move to Domain Referral Transaction Engine.
5tkWinner	apps/disbursementReport	IMicroPrizeService	5 TK micro-prize winner selection; evaluate in Application layer with test cases.
5thWinnerSummary	apps/disbursementReport	IReportingService	Micro-prize report summary; project directly into analytical DTOs.
10tkWinner	apps/disbursementReport	IMicroPrizeService	10 TK micro-prize winner logic; unify with IMicroPrizeService.
ChurnBackCashBack	apps/disbursementReport	IWinBackCampaignService	Win-back campaign reward; move business rules to Domain Strategy Pattern.
5TkCashbackOnNewSub...	apps/report/views.py	ISubscriptionRewardService	Onboarding cashback; trigger via domain event on successful bKash billing.
InsertWinnerForAllEvent	apps/disbursementReport	IWinnerService	Manual winner injection; replace with automated Domain Winner Pipeline.

Continued on next page

Table 16.1 – continued from previous page

Stored Procedure Name	Legacy Call Site	Target .NET 10 Service	Technical Reason for Removal
cms_tournanet_winner	apps/report/views.py	ICmsWinnerService	Admin CMS winner view; execute via paginated EF Core query.
revenue_daily	apps/report/views.py	IRevenueService	Daily revenue summation; calculate via PostgreSQL Materialized Views.
revenue_tournament	apps/report/views.py	IRevenueService	Tournament revenue breakdown; move to Application analytics service.
revenue_wordly	apps/report/views.py	IRevenueService	Wordly revenue breakdown; move to Application analytics service.
new_revenue_report	apps/report/views.py	IRevenueService	Consolidated revenue report; execute on read-replica with caching.
get_revenue_july_report	apps/report/views.py	IRevenueService	Hardcoded month SP; eliminate hardcoded SQL and parameterize date range.
getm_revenue_email_by_date	apps/report/views.py	IEmailNotificationService	Executive email report generator; execute via scheduled Background Worker.
get_renew_rev_email_by_date	apps/report/views.py	IEmailNotificationService	Subscription renewal email report; decouple from DB into .NET BackgroundService.
tournamentTestSp	apps/disbursementReport	- REMOVE -	Dead legacy test procedure; remove completely from production database.

16.5 Clean Architecture Implementation Example: Profile Summary

The code snippet below demonstrates how the high-traffic `ProfileSummaryOneSP` stored procedure is refactored into a high-performance, Redis-cached .NET 10 Clean Architecture service:

Listing 16.1: Refactored ProfileService in .NET 10 Clean Architecture

```

1 public class ProfileService : IProfileService
2 {
3     private readonly IApplicationDbContext _context;
4     private readonly IDistributedCache _cache;
5     private readonly ILogger<ProfileService> _logger;
6
7     public ProfileService(IApplicationDbContext context,
8         IDistributedCache cache, ILogger<ProfileService> logger)
9     {
10         _context = context;
11         _cache = cache;
12         _logger = logger;
13     }
14
15     public async Task<ProfileSummaryDto> GetProfileSummaryAsync(string
16         msisdn, int portalId, CancellationToken ct)
17     {
18         string cacheKey = $"profile:summary:{portalId}:{msisdn}";
19         var cachedData = await _cache.GetStringAsync(cacheKey, ct);
20         if (!string.IsNullOrEmpty(cachedData))
21         {
22             return
23                 JsonSerializer.Deserialize<ProfileSummaryDto>(cachedData)!;
24         }
25     }
26 }

```



```
21     }
22
23     // Clean, optimized EF Core aggregation (replaces legacy
24     ProfileSummaryOneSP)
25     var totalPlayed = await _context.QuizPlays
26         .Where(p => p.Msisdn == msisdn && p.PortalId == portalId
27             && p.IsValid)
28         .CountAsync(ct);
29
30     var avgScore = await _context.QuizPlays
31         .Where(p => p.Msisdn == msisdn && p.PortalId == portalId
32             && p.IsValid)
33         .AverageAsync(p => (double?)p.RightCount, ct) ?? 0.0;
34
35     var totalWins = await _context.DailyWinners
36         .Where(w => w.Msisdn == msisdn && w.PortalId == portalId)
37         .SumAsync(w => (int?)w.PrizeAmount, ct) ?? 0;
38
39     var summary = new ProfileSummaryDto
40     {
41         CompletedQuiz = totalPlayed,
42         AverageQuizScore = Math.Round(avgScore, 2),
43         TotalWinAmount = totalWins,
44         DailyWinningRate = totalPlayed > 0 ?
45             Math.Round((double)totalWins / totalPlayed, 3) : 0.0
46     };
47
48     // Cache summary in Redis with a 5-minute sliding expiration
49     await _cache.SetStringAsync(cacheKey,
50         JsonSerializer.Serialize(summary),
51         new DistributedCacheEntryOptions {
52             AbsoluteExpirationRelativeToNow =
53                 TimeSpan.FromMinutes(5) }, ct);
54
55     return summary;
56 }
```

Chapter Key Takeaways

- ****Database Writer Protection:**** Eliminating 40+ stored procedures liberates primary database CPU, cutting query latency from 4.8s to < 15ms.
- ****Horizontal Scalability:**** Business logic executes within stateless .NET 10 application containers that scale across Kubernetes pods.
- ****Data Locality for Batch Crons:**** Heavy 04:30 AM and 05:30 AM batch rollups over 500,000 rows remain database-local to eliminate gigabytes of unnecessary network egress.
- ****Redis Integration:**** Caching profile summaries and leaderboard snapshots in Redis reduces read load on PostgreSQL by over 85%.

Chapter 17

Technical Glossary and Authoritative Reference Directories

17.1 Authoritative Event and Service ID Directory

The table below documents the production Event ID configuration shared across the Next.js frontend, Django backend, and .NET 10 question engine:

Event ID	Service Type	Frontend Display Title	Game Rules & Configuration
34	Daily	Islamic Daily Quiz Challenge	60 questions, 180s timer, 2 attempts per day.
150	Tournament	Quiz Tournament Arena	Weekly championship tournament rules with cash prizes.
149	BCS	Mind Masters Quiz (BCS Prep)	Subtopics: <code>bcs_prep</code> , <code>bcs_job</code> , <code>bcs_class8</code> .
75	Sports	Sports Quiz Arena	Football, Cricket, and Sports trivia categories.
134	SpellingBee	Spelling Bee Challenge	English & Bengali vocabulary spell-check puzzles.
67	Wordly	Wordly Bangla	5-letter daily Bengali hidden word puzzle.
79	Wordly	Wordly English	5-letter daily English hidden word puzzle.
151	MemoryMatch	Memory Match Arcade	Visual card-matching reflex game (Easy, Medium, Hard).
152–155	Sudoku	Sudoku Master Series	Sudoku levels: Easy (152), Medium (153), Hard (154), Expert (155).

Table 17.1: Quizard Authoritative Event and Service ID Directory

17.2 Authoritative Multi-Tenant Portal Directory (PortalType)

Quizard provides strict multi-tenant isolation across telecom carriers and digital wallet partners:

17.3 Glossary of Core Domain Entities

MSISDN: Mobile Station International Subscriber Directory Number (the player’s unique mobile phone number in Bangladesh, e.g., 01711000001).

AES-256-CBC: Advanced Encryption Standard with 256-bit key length operating in Cipher Block Chaining mode, securing all client score transmissions.

Dense Rank: A database ranking function where identical scores receive identical rank numbers without creating gaps in consecutive rank assignments.

Portal ID	Enum Key	Operator / Channel	Tenant Architecture & Billing Integration
15	Quizard	bKash / Quizard Web Core	Primary parent company; bKash tokenized MFS charging.
16	ApplinkBL	Banglalink (BL)	Direct Operator Billing via Banglalink Applink gateway.
17	DigimartGP	Grameenphone (GP)	Direct Operator Billing via GP Digimart subscription SDK.
18	Wordly	bKash Wordly	Standalone Wordly puzzle tenant; dedicated subscription pass.
19	EngageX	EngageX Promotional	Always-subscribed promotional and partner testing tenant.
20	AppInApp	bKash AppInApp	Micro-frontend embedded inside official bKash mobile app.
21	ShadhinMusic	Robi / Shadhin Music	Co-branded portal mapped to Robi telco billing backend.
22	Arcade	Robi Wordly / Arcade	Robi arcade gaming portal mapped to Wordly company rules.
25	GPArCADE	Grameenphone Arcade	GP arcade subscription portal for casual puzzle series.

Table 17.2: Quizard Authoritative Portal ID and Carrier Gateway Matrix

Velocity Check: Anti-cheat validation ensuring the rate of answered questions does not exceed human biological reaction thresholds (≤ 1.5 answers/sec).

Final Architecture Conclusion

Quizard's architecture represents a gold standard for high-performance, real-time gaming in emerging markets. By migrating from Django/MySQL to high-throughput .NET 10/PostgreSQL, coupling dynamic rules governance, sub-50ms question sampling, end-to-end cryptographic integrity, and automated financial verification, the platform delivers an engaging and trustworthy experience to millions of contestants.